

A Survey of Agent Skills: Toward Procedural Infrastructure for LLM Agents

CEHAO YANG^{1,2,3*}, XIAOJUN WU^{1,2,3*}, HONGHAO LIU^{1,2,3*}, XUEYUAN LIN^{1,2*}, CHENGJIN XU^{1,3}, XUHUI JIANG^{1,3}, YUANLIANG SUN^{1,3}, WENJIE ZHANG^{1,3}, ZHICHAO SHI^{1,3}, YIJIE XU², JIA LI², HUI XIONG^{2†}, JIAN GUO^{1†},

¹IDEA Research, International Digital Economy Academy, China

²The Hong Kong University of Science and Technology (Guangzhou), China

³DataArc Tech Ltd, China

Abstract

Large language model (LLM) agents are increasingly expected to solve long-horizon tasks through repeated interaction with external environments, tools, users, and other agents. In this setting, agent skills have emerged as a central mechanism for transforming fragmented experience into reusable procedural knowledge. Unlike raw memory which preserves past traces, or abstract rules which often lack executable detail, skills compress recurring patterns of successful behavior into operational artifacts that can guide future action. However, existing studies on agent skills remain scattered across diverse areas, making it difficult to form a unified understanding of what skills are, how they are represented, and how they should be governed. To bridge this gap, this paper presents a comprehensive survey of agent skills. **We propose a six-layer taxonomy covering ontology, representation and packaging, lifecycle, runtime integration, governance, and applications.** We first define agent skills as reusable procedural abstractions that connect memory, human expertise, and execution. We then review major skill representations, including natural-language guidance, executable code snippets, decision graphs, filesystem-based packages, and structured skill records. Next, we analyze the skill lifecycle, from acquisition, storage, retrieval, usage, and maintenance to selective internalization into model behavior. We further examine how skills integrate with terminal interfaces, tool interfaces, multi-agent systems, and agent harnesses. In addition, we discuss the emerging skill ecosystem, its security and governance risks, and mechanisms for trusted deployment. Finally, we survey applications in robotics, games, web agents, GUI/mobile/OS agents, and software engineering, and identify open challenges in evaluation, robustness, standardization, safety, and infrastructure. Related resources, as well as the latest developments in this field, are accessible on <https://github.com/DataArcTech/Awesome-Agent-Skill-Papers>.

*These authors contributed equally to this research.

†Corresponding author.

1 Introduction

Large language models (LLMs) are increasingly used not only as conversational systems, but as agents that plan, call tools, operate files, inspect web pages, write code, and execute long-horizon workflows [69, 112]. This shift is accelerated by recent frontier agent harnesses such as Claude Code, Codex, OpenClaw, and Hermes Agent that turn model outputs into situated action loops over files, tools, terminals, browsers, and external services [6, 79, 81, 83]. In other words, practical agent engineering has moved through a recognizable arc: from *prompt engineering*, to *context engineering*, and now toward *harness engineering* [161].

This arc reflects a migration of capability from the model’s weights into external structures around the model. Agent systems relocate these burdens into memory stores, reusable skills, interaction protocols, and the harness that coordinates them [161]. Among these externalized structures, *skills* have recently emerged as a particularly important abstraction. Anthropic’s Claude Skills introduced a concrete and widely discussed implementation: skills are “folders of instructions, scripts, and resources” that harness loads dynamically to improve specialized-task performance [8]. ***Agent skills are rapidly becoming a new ecosystem layer which is already substantial.*** Industry reports list multiple fast-growing marketplaces: At April 2, 2026, SkillsMP claims more than 425,000 skills, LobeHub Skills lists 169,739 skills, agentskill.sh lists more than 110,000 skills, skills.sh tracks around 87,000 unique skills, and ClawHub registers more than 20,000 skills¹.

As skill marketplaces expand, they increasingly resemble app stores or open package registries, inheriting many of the same risks of abuse, malicious uploads, and supply-chain attacks. The user-uploaded skills in OpenClaw marketplaces became an attack surface for credential theft and malware distribution, forcing platforms to introduce stricter publication thresholds and reporting mechanisms². The skill marketplaces create forms of “implicit trust,” where users rely on names, descriptions, popularity signals, and category labels rather than auditing the underlying code itself³. In such environments, malicious skills can masquerade as productivity or security tools.

These challenges shift our attention from the practical question of how to write a useful skill to a more fundamental question: ***What is the nature of an agent skill?*** To clarify this emerging abstraction, we provide a comprehensive survey of agent skills from a layered perspective. Rather than treating skills as a single artifact type, we organize the field into six connected layers: ontology, representation, lifecycle, runtime integration, governance, and applications.

- At the ***ontology layer*** in Section 2, we clarify the conceptual status of agent skills and explain why they are becoming a distinct object of study in LLM-based agents. This layer asks what skills are, what problems they solve, and how they differ from adjacent notions such as episodic memory, abstract rules, external tools, and general task instructions.
- At the ***representation layer*** in Section 3, we examine how agent skills are encoded, packaged, and made available for reuse. Existing systems represent skills in multiple forms, ranging from lightweight natural-language guidance and heuristic rules to executable code snippets, decision graphs, filesystem-based packages, and structured skill records. These representations differ in their flexibility, executability, inspectability, and suitability for composition.
- At the ***lifecycle layer*** in Section 4, we analyze how skills evolve over time rather than treating them as static artifacts. A complete skill lifecycle includes acquisition from trajectories, external resources, or human expertise; storage in repositories that support later discovery and composition; retrieval and usage under context and execution constraints; refinement through feedback, evaluation, and failure diagnosis; deprecation or retirement when skills

¹<https://www.kdnuggets.com/top-5-agent-skill-marketplaces-for-building-powerful-ai-agents>

²<https://www.theverge.com/news/874011/openclaw-ai-skill-clawhub-extensions-security-nightmare>

³<https://www.lakera.ai/blog/the-agent-skill-ecosystem-when-ai-extensions-become-a-malware-delivery-channel>

become redundant or harmful; and selective internalization into model behavior when a skill becomes sufficiently reliable and general.

- At the **runtime integration layer** in Section 5, we study how skills become active components of agent execution. Skills must interface with concrete runtime environments such as terminals, tool APIs, browsers, filesystems, multi-agent frameworks, and agent harnesses. In this layer, a skill acts as a procedural adapter between high-level user intent and low-level executable actions: it helps the agent decide which commands to run, which tools to call, how to bind parameters, how to interpret intermediate results, and how to verify completion.
- At the **governance layer** in Section 6, we investigate the risks and control mechanisms that arise when skills are created, shared, installed, and executed at scale. As skill repositories and marketplaces grow, skills begin to resemble software packages: they can be versioned, distributed, reused, composed, and potentially abused. This introduces security and governance concerns such as malicious skills, prompt-injection payloads, hidden dependencies, credential leakage, excessive permissions, and supply-chain attacks.
- At the **application layer** in Section 7, we survey how agent skills support concrete downstream domains. Across these domains, skills serve as reusable procedural infrastructure that improves generalization, reduces repeated planning effort, and makes agent behavior more modular and inspectable.

Previous Surveys	L1 Ontology	L2 Repres.	L3 Lifecycle	L4 Runtime	L5 Governance	L6 Application
Procedural Memory [121]	○	●	-	○	-	○
Architecture, Acquisition [131]	-	○	●	○	○	-
SoK: Beyond Tool Use [42]	○	●	●	○	●	-
Externalization (Skill) [161]	-	○	○	●	-	-
Recipes for Agents [129]	●	-	-	-	●	-
Ours	●	●	●	●	●	●

Table 1. Coverage of five recent discussions, perspectives or surveys on agent skills across our proposed six-layer taxonomy: (L1) Ontology, (L2) Representation, (L3) Lifecycle, (L4) Runtime Integration, (L5) Governance, and (L6) Applications. ● indicates primary focus, ○ indicates partial coverage, and - indicates limited or indirect discussion.

Compared with these five recent discussions, perspectives, and surveys on agent skills, our survey provides broader and more balanced coverage across the proposed six-layer taxonomy, as summarized in Table 1. Through this survey, we aim to offer both a historical map of how agent skills have evolved and a system-level framework for understanding the role they play in the next generation of LLM-based agents.

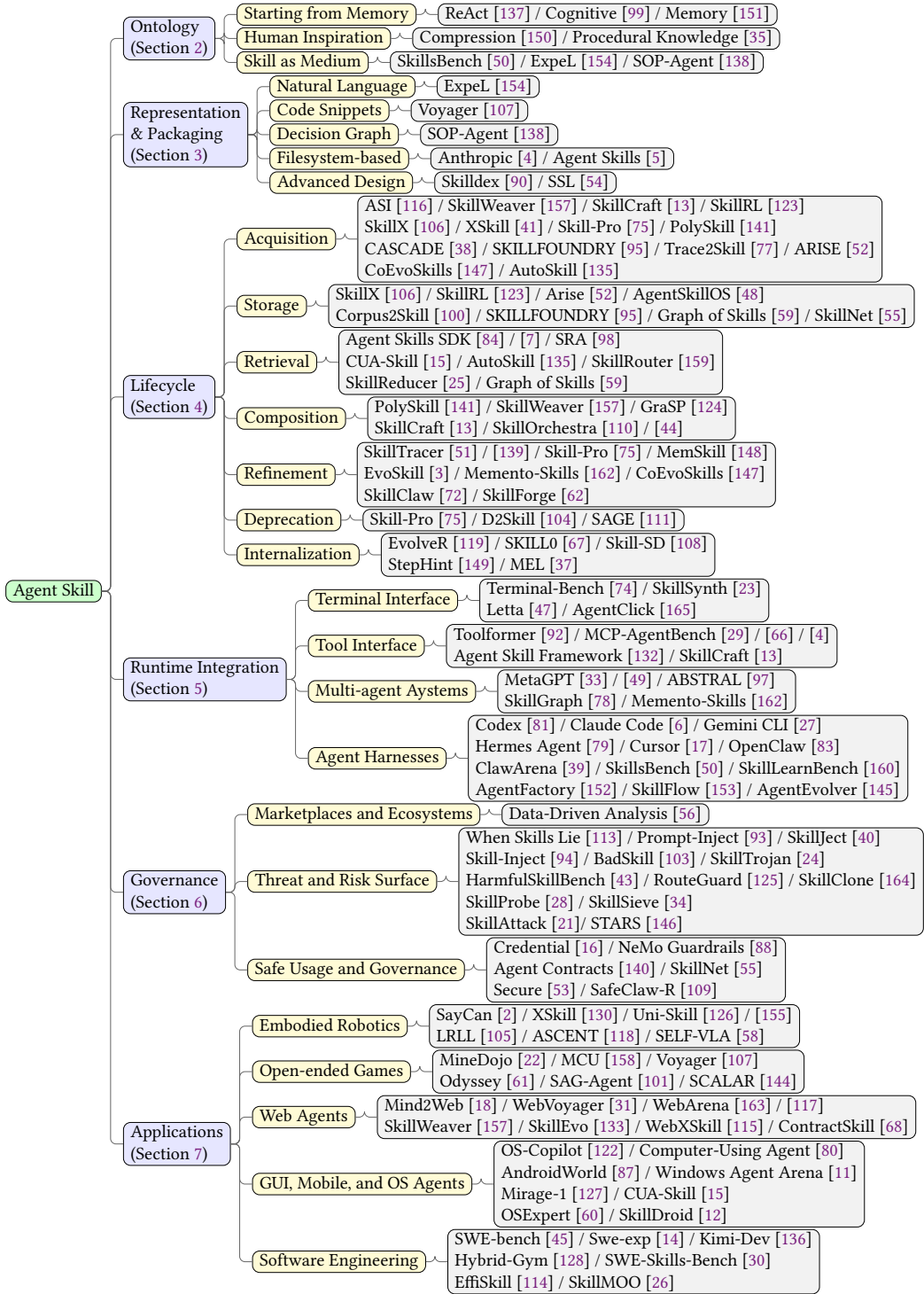


Fig. 1. The taxonomy of agent skills.

CONTENTS

Abstract 1

1 Introduction 2

2 Layer 1: Ontology 6

2.1 Starting from Memory: The Cognitive Architecture of Agents 6

2.2 Human Inspiration: Skill as Compression from Memory to Pattern 7

2.3 Skill as a Medium: Distilling and Reusing Human Wisdom 8

3 Layer 2: Representation and Packaging 10

3.1 Natural Language 10

3.2 Code Snippets 11

3.3 Decision Graph 11

3.4 Filesystem-based 12

3.5 Advanced Design 13

4 Layer 3: Lifecycle 14

4.1 Acquisition Stage 14

4.2 Storage Stage 16

4.3 Usage Stage 18

4.4 Maintenance Stage 19

4.5 Internalization Stage 22

5 Layer 4: Runtime Integration 23

5.1 Interface Accessing 23

5.2 System Integration 26

6 Layer 5: Governance 28

6.1 Marketplaces and Ecosystems 28

6.2 Threat and Risk Surface 30

6.3 Safe Usage and Governance Mechanisms 31

7 Layer 6: Application 32

7.1 Embodied Robotics & Physical Manipulation 32

7.2 Open-Ended Game Environments 33

7.3 Web Agents & Browser Automation 35

7.4 GUI, Mobile & OS Agents 35

7.5 Software Engineering & Coding Agents 36

8 Challenges and Future Opportunities 38

8.1 Measuring Skill Value Beyond Task Success 38

8.2 The Skill–Context Trade-off 38

8.3 Skill Interference and Behavioral Conflict 38

8.4 The Boundary Between Adaptation and Drift 39

8.5 Skills as Infrastructure, Not Add-ons 39

9 Conclusion 40

References 41

2 Layer 1: Ontology

“Art arises when from many notions gained by experience one universal judgement about a class of objects is produced.”

— Aristotle

The originality of agent skills begins with a basic question: **what kind of knowledge allows an agent to act competently across repeated situations?** An agent does not merely answer isolated prompts. It observes an environment, reasons over the current state, selects actions, receives feedback, and gradually accumulates traces of interaction. These traces contain useful information, but in their raw form they are too detailed, too large, and too situation-specific to serve as efficient guidance for future behavior. A skill can therefore be understood as a compressed and reusable form of agent experience: it transforms many particular episodes into a procedural pattern that can be invoked when a similar class of situations appears.

2.1 Starting from Memory: The Cognitive Architecture of Agents

A language agent can be described as a system that places a language model inside an interactive loop. At each step, the agent receives observations, maintains an internal state, reasons about possible next actions, and acts on the external environment [137]. In a cognitive architecture, this loop is not only an input-output process, it also involves internal storage, retrieval, planning, and state update mechanisms [99].

Memory is the first layer because it determines what information remains available to the agent. An agent’s short-term memory is the content currently available in the prompt or context window. However, it is also limited in capacity and duration. Long-term memory addresses this limitation by storing information outside the immediate context window. It may include past interaction records, user preferences, domain knowledge, project conventions, failure cases, successful solutions, and other persistent artifacts. Unlike short-term, long-term memory is not always visible to the model. Thus, long-term memory is not simply a larger context window; it is a managed storage system that mediates between accumulated experience and present action [151].

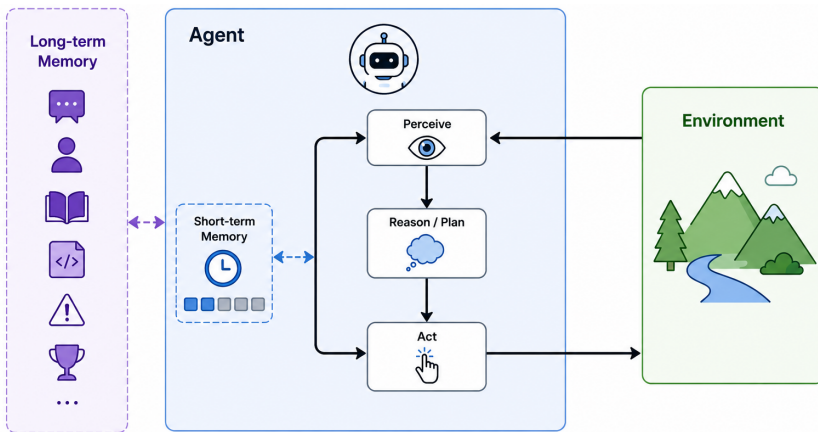


Fig. 2. The cognitive architecture of agents from the memory system perspective [151].

2.2 Human Inspiration: Skill as Compression from Memory to Pattern

Human skill learning provides a useful analogy. A person who learns to cook, debug code, negotiate, or diagnose a device does not retain every past episode as an independent instruction. Instead, repeated experience produces patterns. The learner begins to recognize relevant cues, ignore irrelevant details, sequence actions efficiently, anticipate common failures, and verify whether a step has succeeded. Through this process, many concrete experiences are compressed into a more general procedural form.

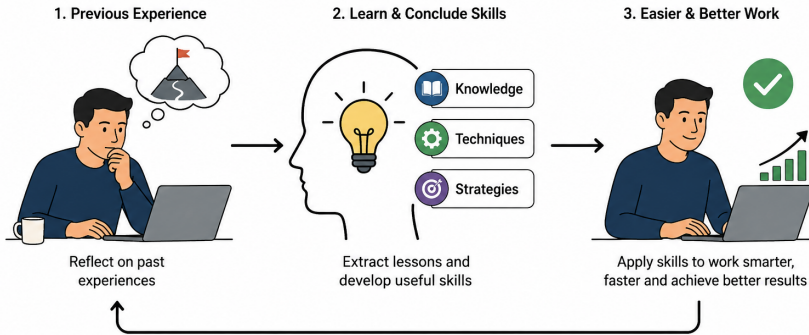


Fig. 3. A case of human skill learning. The person learns from repeated experience, extracting lessons and patterns to develop skills that can guide future behavior.

Key Takeaways

The same principle applies to agents. Raw trajectories contain observations, actions, errors, corrections, and outcomes. Episodic memories preserve parts of these trajectories, but they still remain close to the original experience. Skills introduce a higher level of compression. They **extract the recurring structure of successful behavior**: when the pattern is relevant, what subgoals should be pursued, which tools or resources should be used, what order is usually effective, and how success should be checked. In this sense, a skill is procedural knowledge derived from experience.

This view places skills between episodic memory and abstract rules. An episodic memory is highly specific: it records what occurred in a particular task. A rule is highly compressed: it states a general principle, often without detailed operational guidance. A skill lies between them. It is more general than a single memory, because it applies across a class of tasks; yet it is more operational than a rule, because it contains an actionable procedure. This middle position makes skills especially important for long-horizon agents, where raw experience must be condensed into reusable guidance without losing practical detail [150].

The benefit of this compression is not only storage efficiency. **A well-formed skill can reduce planning burden, stabilize behavior, and make execution more predictable.** Instead of reconstructing a procedure from scratch at every encounter, the agent can retrieve an existing behavioral pattern and adapt it to the present case. Prior work on procedural knowledge shows that explicit procedural structure can improve agentic workflows by helping the agent maintain task progress, respect dependencies among steps, and verify completion before moving forward [35]. Thus, skills function as cognitive shortcuts: they convert accumulated experience into structured action.

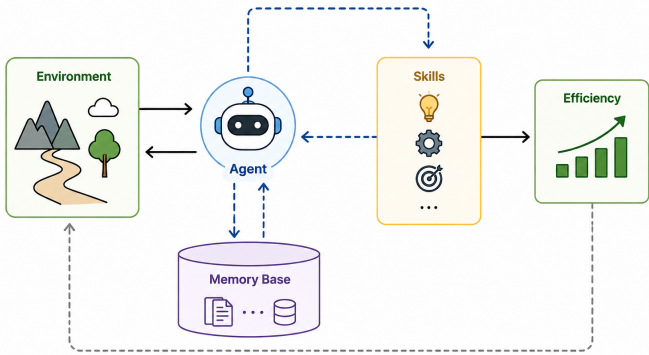


Fig. 4. A case of agent skill learning. The agent interacts with the environment and accumulates experience. The skill extracts patterns from the experience and guides future behavior.

2.3 Skill as a Medium: Distilling and Reusing Human Wisdom

Skills are not limited to patterns discovered from an agent’s own trajectories. They can also encode human wisdom. Human experts often possess procedural knowledge that is difficult to express as isolated facts: how to handle exceptions, how to choose among tools, how to follow domain conventions, how to avoid common mistakes, and how to evaluate whether an intermediate result is acceptable. When such knowledge is written as reusable procedural guidance, it becomes a skill-like artifact for the agent.

This matters because agents may not reliably generate high-quality skills by themselves. Self-generated procedural knowledge can be incomplete, overly broad, brittle, or misaligned with real task requirements. Empirical studies of agent skills suggest that curated skills can substantially improve performance, while self-generated skills may provide inconsistent benefit across tasks and domains [50]. Human-provided procedural knowledge is therefore not merely an optional supplement; it can serve as an important source of quality control, domain grounding, and task-specific reliability.

Agent skills also provide a medium through which human knowledge and agent experience can be combined. On one side, agents can summarize past trials into reusable insights, preserving lessons from success and failure without changing model parameters [154]. On the other side, humans can provide domain-specific procedures that guide agents through specialized workflows [138]. The resulting skill is neither pure memory nor pure instruction. It is an operational artifact that connects experience, expertise, and execution.

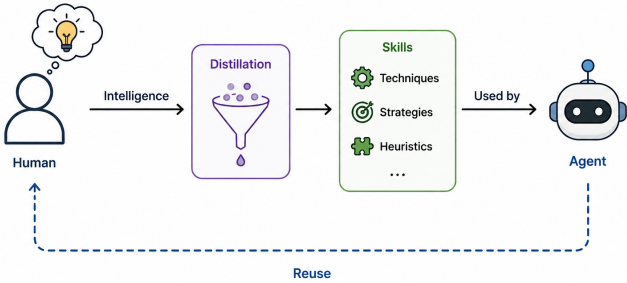


Fig. 5. Skill as a medium connecting human wisdom and agent experience.

As skill libraries grow, skills become part of a broader ecosystem. They can be authored, revised, selected, shared, evaluated, and composed. This ecosystem makes agent behavior more modular and inspectable: instead of embedding every behavioral assumption inside a long prompt or opaque model parameter, a skill externalizes a piece of procedural knowledge in a form that can be reviewed and improved [56].

Key Takeaways

Experience produces traces. Memory stores selected traces and facts. Skills compress repeated experience and human expertise into reusable procedures. Rules further abstract these procedures into general principles. Among these forms, **skills are the central bridge between knowing and doing: they translate accumulated knowledge into structured behavior** that an agent can apply in future tasks.

The preceding discussion motivates a more precise formulation of skill. A skill must have identifiable boundaries: it should indicate when it applies, what procedure it provides, what resources it depends on, and how the agent can determine whether using it has succeeded. The following definition makes this structure explicit by representing a skill as a reusable unit of procedural knowledge with four core components.

A Formal Definition of Agent Skill

An agent skill is a reusable unit of procedural knowledge that compresses repeated experience, human expertise, or both into an actionable pattern for competent behavior across a class of tasks. It specifies when the pattern is relevant, what subgoals should be pursued, what actions, tools, or resources are typically useful, in what order they should be applied, and how successful execution should be checked. A formal definition of agent skill can be written as follows:

$$s = \langle C, \mathcal{P}, \mathcal{R}, \mathcal{E} \rangle \quad (1)$$

where

- C is the **condition of applicability**, which specifies when the skill should be used, including task type, context, input pattern, environmental state, or goal.
- $\mathcal{P}$ is the **procedural core**, which describes what the agent should do, such as a sequence of steps, a decision graph, a natural-language heuristic, an executable program, or a filesystem-based package.
- $\mathcal{R}$ is the **resource interface**, which specifies the tools, files, memories, scripts, references, or external resources that the skill may rely on.
- $\mathcal{E}$ is the **evaluation or verification criterion**. It describes how the agent checks whether the skill has been applied successfully, including completion conditions, quality checks, error signals, or feedback.

When an agent policy π interacts with an environment through a trajectory $\tau = (o_t, z_t, a_t)_{t=1}^T$ of observations, thoughts and actions [137], a skill s is invoked when its applicability condition is satisfied, i.e., $o_t \in C$. Once invoked, the agent follows the procedural core $\mathcal{P}$ under policy π , accesses the resources specified by $\mathcal{R}$, and uses the execution criterion $\mathcal{E}$ to determine whether execution has succeeded, should continue, or requires revision.

3 Layer 2: Representation and Packaging

In this section, we survey how skills are represented and packaged for reuse in LLM agents. We first examine natural-language abstractions, which encode reusable experience as textual guidance in Section 3.1. We then discuss code snippets as executable skill artifacts in Section 3.2, followed by decision graphs that make procedural control flow explicit in Section 3.3. Section 3.4 turns to Claude-style filesystem packages that combine metadata, instructions, scripts, and auxiliary resources under progressive disclosure. Finally, Section 3.5 surveys advanced representations that add package-level management and structured intermediate forms for discovery, governance, and composition.

3.1 Natural Language

Skill for ALFWorld 1. Always confirm the presence of an item before attempting to interact with it. 2. Before using an appliance or storage, ensure it is ready for use. If not, take necessary actions to prepare it. Once an item is found and prepared (if necessary), it should be placed in the designated location immediately to avoid confusion. 19. When a task involves multiple items of the same type, ensure to keep track of the number of items already found and interacted with to avoid confusion and ensure task completion. 20. When a task requires heating or cooling an item, ensure to use the appropriate appliance such as a microwave for heating or a fridge for cooling.	Skill for HotpotQA 1. Always ensure to exhaust all possible search queries related to the task at hand before concluding. 2. When searching for information, consider using different combinations of keywords, and if necessary, break down complex queries into simpler, more specific ones to increase the chances of finding the required information. 12. When searching for specific episodes or events, include the name of the series or event in the search query to increase the chances of finding the required information. 13. When comparing the attributes of multiple entities, ensure to keep a clear record of each entity's attributes to avoid confusion and ensure accurate comparison.
Skill for WebShop 1. Before purchasing, verify that the product meets all the specific requirements mentioned in the task instruction. 2. Always ensure to understand the product attributes (like color, size, etc.) mentioned in the product description, and select them if they are selectable options to ensure the product matches the task instruction. 19. If the first product selection attempt is unsuccessful, do not repeat the same search query. Instead, consider other products from the initial search results or modify the search query to get more relevant results. 20. If the product has selectable options, ensure to select the correct options that match the task instruction before purchasing.	Skill for FEVER In order to successfully complete factual verification tasks, begin by clearly understanding the claim. Then formulate a search query that is precise and directly related to the claim. Include the main subjects or context from the claim in your query. If the initial search doesn't yield desired results, consider refining the query, using synonyms, or breaking down the claim into smaller parts. When you're ready to draw a conclusion, double-check it against the information obtained and ensure its accuracy. Lastly, always be prepared to exhaust all possible search queries related to the task at hand before concluding. Remember, the claim can either be supported, refuted, or there might not be enough information to draw a conclusion.

Fig. 6. A case of natural-language skill representation [154]. The skill is a reusable insight extracted from past experience, expressed as a compact textual rule, heuristic, or guidance. The agent can retrieve and apply this insight across different tasks that share similar reasoning patterns.

The simplest representation treats a skill as a reusable natural-language abstraction over past experience. In this form, the skill is not an executable object but a compact textual rule, heuristic, or insight derived from trajectories of interaction. A natural-language skill can then be represented as an extracted insight

$$s_{NL} = f_{abstraction}(\mathcal{B}), \quad (2)$$

where $\mathcal{B} = \{\tau_1, \dots, \tau_n\}$ is a pool of successful and failed experiences, and f_{NL} abstracts recurring patterns into verbal guidance. Such guidance may describe what to try, what to avoid, how to decompose a task, or which observations should trigger a particular strategy.

This representation is lightweight and flexible. It can be inserted directly into the agent's prompt, retrieved as a memory item, or combined with similar past trajectories during inference. Its main advantage is semantic generality: a single insight may apply across many tasks whose surface forms differ but whose reasoning pattern is similar. However, because the procedure remains expressed in ordinary language, execution still depends on the agent's interpretation. The representation therefore supports broad transfer but provides limited guarantees about control flow, preconditions, or side effects [154].

3.2 Code Snippets

Code Snippets	Description
<pre> async function combatZombie(bot) { // Equip a weapon const sword = bot.inventory.findInventoryItem(mcData.itemsByName["stone_sword"],).id }; if (sword) { await bot.equip(sword, "hand"); } else { await craftStoneSword(bot); ... } // Craft and equip a shield ... // Recover hunger ... // Look for and combat a zombie ... } </pre>	<pre> async function combatZombie(bot) { // The function is about equipping a stone sword to combat a zombie. If a stone sword is not found, it will craft one. Additionally, it crafts and equips a shield for added protection. Afterwards, it proceeds to cook sticks in order to restore hunger. Once hunger is replenished, it actively searches for a zombie and engages in combat with it. } </pre>

Fig. 7. A case of code-snippet skill representation [107]. The skill is an executable program that encapsulates reusable behavior. The agent can retrieve and execute this program directly or compose it with other programs to solve more complex tasks.

A more operational representation encodes a skill as executable code. A code-based skill can be written as

$$s_{\text{code}} = \langle d, p, e \rangle, \quad (3)$$

where d is a natural-language description used for retrieval, p is an executable program, and e is meta parameter list used to configure the program. The description d functions as the indexing key, while p functions as the reusable behavior.

This representation is especially suitable for temporally extended and compositional actions. A function such as `craftStonePickaxe()` or `combatZombieWithSword()` can encapsulate many low-level actions behind a single callable interface. When a new task is encountered, the agent can retrieve related programs by matching the task description and current context against stored descriptions. The retrieved programs can then be reused directly or composed into more complex programs. The code-snippet representation improves reproducibility and compositionality because the behavior is grounded in an executable artifact. Its limitation is that correctness depends on the reliability of generated code and on the availability of an execution environment in which the program can be tested [107].

3.3 Decision Graph

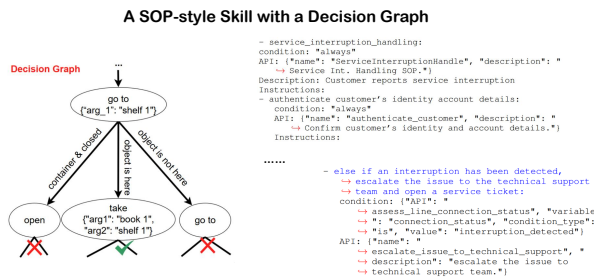


Fig. 8. A case of decision-graph skill representation [138]. The skill is a structured workflow represented as a directed graph, where nodes denote candidate actions or procedures and edges denote conditional transitions.

A SOP-style structured workflow can be represented as a decision graph. Let $G = (V, E)$ be a directed graph, where each node $v \in V$ denotes a candidate action or procedure, and each edge $e = (v_i, v_j) \in E$ is labeled by a condition κ_e . The condition may be an explicit IF predicate or an unconditional ALWAYS transition. An execution trace is then a path or subtree traversal through G , selected according to the observations available at runtime.

In this representation, a skill is not a single instruction but a controlled decision process:

$$s_{\text{graph}} = \langle G, v_0, \mathcal{A}, \mathcal{K} \rangle, \quad (4)$$

where v_0 is the entry node, $\mathcal{A}$ is the set of available actions or function calls, and $\mathcal{K}$ is the set of branch conditions. At each step, the agent evaluates the relevant conditions, selects valid branches, and executes the corresponding actions. The graph can express sequential execution, branching, loops, and cascaded subprocedures.

The decision-graph representation is well suited for domain-specific standard operating procedures. It constrains the action space by exposing only the function calls relevant to the current node or branch, thereby improving controllability and reducing unnecessary exploration. Compared with free-form natural language, it makes the workflow more inspectable; compared with a monolithic program, it preserves the high-level procedural logic in a form that can be edited by non-programmers. Its main cost is authoring and maintenance: the graph must be logically complete, and ambiguous branch conditions may still lead to incorrect traversal [138].

3.4 Filesystem-based

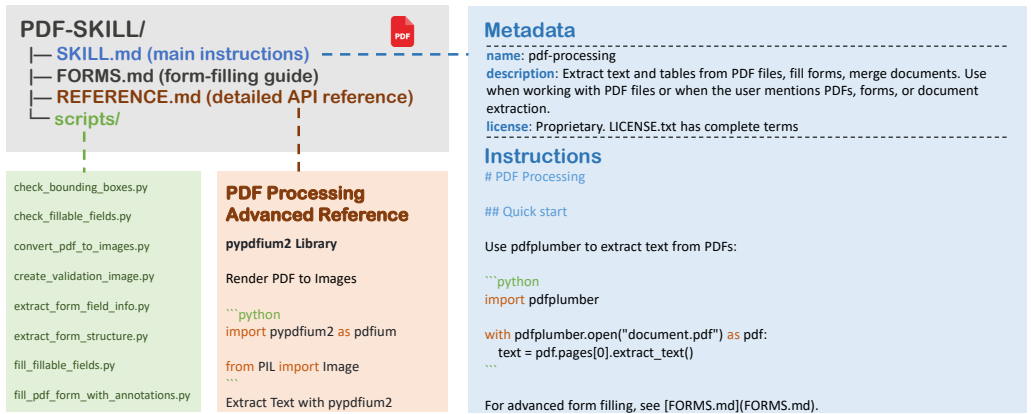


Fig. 9. A case of filesystem-based Claude-style skill representation [4]. The central artifact is a SKILL.md file containing metadata and instructions, optionally accompanied by scripts, templates, references, and other resources. The agent progressively loads these files as needed during task execution.

A filesystem-based representation packages a skill as a directory. The central artifact is a SKILL.md file containing metadata and instructions, optionally accompanied by scripts, templates, references, and other resources. A skill in this style can be formalized as

$$s_{\text{fs}} = \langle m, I, C, R \rangle, \quad (5)$$

where m is metadata, I is the main instruction body, C is executable code (optional), and R is a set of auxiliary resources (optional). The metadata requires a name and description, which allow the agent to decide when the skill should be invoked.

This representation treats a skill as an onboarding package rather than a single prompt. It supports modularity, reuse, and context efficiency because large resources can remain outside the context window until they are relevant. It also separates flexible natural-language guidance from deterministic helper scripts. The same package can therefore include both procedural instructions and executable utilities, while the agent controls which parts are brought into context during task execution [4, 5].

3.5 Advanced Design

One advanced skill representation extends a skill from a single instruction file into a managed package. In this view, a skill is treated similarly to a software package: it has metadata, version information, installation scope, validation rules, and registry support. A package manager can install, remove, update, search, publish, and validate skills, while a registry makes skills discoverable across users and projects. It also introduces scoped distribution, where skills can be installed globally, shared across teams, or kept local to a project. Such scoping helps control which skills are available in a given context and prevents unrelated skills from unnecessarily entering the agent’s working environment. A further extension is the skillset, which bundles related skills together with shared assets such as templates, reference documents, vocabularies, or conventions. This makes the behavior of related skills more consistent than installing each skill independently [90].

Another advanced direction is to convert text-heavy skill documents into structured records. Instead of leaving all useful information embedded in natural language, the skill is separated into three layers: scheduling, structural, and logical. The scheduling layer describes when the skill should be used, including its intent, inputs, outputs, and goal-level tags. The structural layer describes the execution flow, such as the phases or scenes that the agent should follow. The logical layer records concrete actions and resource usage, such as reading files, calling tools, writing outputs, using network resources, or producing side effects. This representation makes skills easier to search, inspect, compare, and assess for risk because the important operational details are no longer hidden inside prose. It does not replace the original skill document; rather, it provides a machine-readable layer that can support skill discovery, selection, reuse, composition, and governance [54].

4 Layer 3: Lifecycle

The lifecycle layer describes how agent skills move from episodic experience to reusable, governable, and eventually internalized competence. The lifecycle is decomposed into five coupled stages: acquisition, where raw trajectories and external resources are transformed into reusable abstractions (Section 4.1); storage, where skills are structured for later discovery and composition (Section 4.2); usage, where retrieved skills are disclosed, orchestrated, and executed under runtime constraints (Section 4.3); maintenance, where skills are monitored, refined, evolved, deprecated, or retired according to empirical utility (Section 4.4); and internalization, where mature external skills are selectively compressed into intrinsic model behavior (Section 4.5). As shown in Figure 10, these stages frame agent skills not as static prompt artifacts, but as evolving operational knowledge that circulates between experience, memory, execution, and learning.

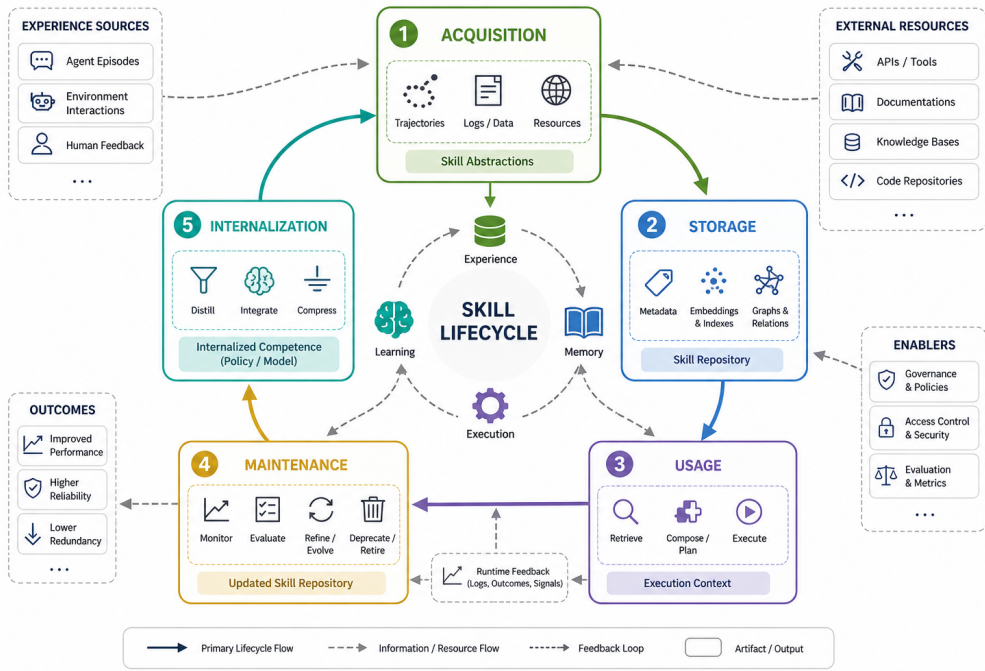


Fig. 10. An overview of the lifecycle of agent skill. The lifecycle management focuses on the temporal processes that make skill repositories useful over repeated deployment: skills must first be discovered, then organized, invoked, maintained, and selectively absorbed into the agent policy.

4.1 Acquisition Stage

The acquisition stage concerns how an agent transforms situated experience into reusable procedural abstractions. Although recent studies consider different environments, they largely instantiate a shared acquisition pattern: the agent interacts with an environment, produces trajectories, receives success or failure signals, and abstracts recurring or causally useful structures into skills [13, 106, 116, 123, 134, 157]. Skill acquisition is not merely memory accumulation, rather, it is a process of converting episodic interaction traces into compact, reusable, and executable behavioral knowledge.

Formally, we consider an environment

$$\mathcal{E} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}), \quad (6)$$

where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, $\mathcal{T}$ the transition dynamics, and $\mathcal{R}$ the reward or evaluation signal. An agent acts according to a policy $\pi(a_t \mid s_0, a_0, \dots, s_t)$. A rollout generates a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$ which can be evaluated by a binary success label $y(\tau) \in \{0, 1\}$, a cumulative return $R(\tau)$, or richer feedback from verifiers, critics, users, or environment-specific evaluators. The purpose of acquisition is to induce a set of skills $\Sigma = \{\sigma_1, \dots, \sigma_K\}$ that improves future behavior beyond the specific episodes from which those skills were derived. The central operation of acquisition can therefore be written as a discovery operator

$$\mathcal{F}_{\text{disc}} : \mathcal{D} \rightarrow \Sigma, \quad (7)$$

where $\mathcal{D} = \{\tau_i\}_{i=1}^N$ is a collection of trajectories or experience records. This operator compresses raw experience into reusable structures while filtering out incidental details. One common acquisition regime is success-driven discovery, in which skills are extracted primarily from successful trajectories:

$$\Sigma = \mathcal{F}_{\text{disc}}(\{\tau \in \mathcal{D} : y(\tau) = 1\}). \quad (8)$$

This regime assumes that successful executions contain reusable procedural regularities, such as recurring navigation routines, tool chains, search-filter-select patterns, or multi-step workflows [13, 116, 157]. Its strength lies in extracting compact procedures from demonstrated competence. Its limitation is that success alone often leaves implicit the boundary conditions under which a procedure fails.

A complementary regime treats acquisition as a contrastive inference problem over both successful and failed trajectories:

$$\Sigma = \mathcal{F}_{\text{disc}}(\mathcal{D}^+, \mathcal{D}^-), \quad (9)$$

where $\mathcal{D}^+$ and $\mathcal{D}^-$ denote positive and negative experience sets. In this view, failure is not discarded as noise; it provides evidence about missing preconditions, brittle assumptions, ineffective action choices, and necessary recovery strategies. Such contrastive acquisition is especially important in long-horizon and reinforcement-learning settings, where the value of a skill depends not only on whether it can reproduce a successful behavior, but also on whether it improves policy learning, reduces exploration cost, and remains useful under distribution shift [41, 75, 104, 120, 123]. This line of work suggests that robust skills are often best understood as abstractions over differences between effective and ineffective behavior, rather than summaries of success alone.

A third regime emphasizes exploration-driven acquisition. Instead of assuming that the available trajectory set already contains the relevant skills, the agent actively expands the experience distribution from which skills can be learned:

$$\tau \sim \pi_{\text{explore}}(\cdot), \quad \Sigma \leftarrow \mathcal{F}_{\text{disc}}(\mathcal{D} \cup \{\tau\}). \quad (10)$$

This approach is crucial when the environment contains latent affordances, underexplored task types, or feasible behaviors that are not present in the initial data. Some systems generate environment-grounded tasks through exploration and feedback, while others mine procedural knowledge from external resources such as documentation, scripts, notebooks, APIs, papers, and repositories [38, 57, 95, 134, 141]. These studies broaden the notion of acquisition: skills need not be distilled only from completed task trajectories; they may also emerge from active curriculum construction, self-reflection, resource mining, and tool-mediated investigation.

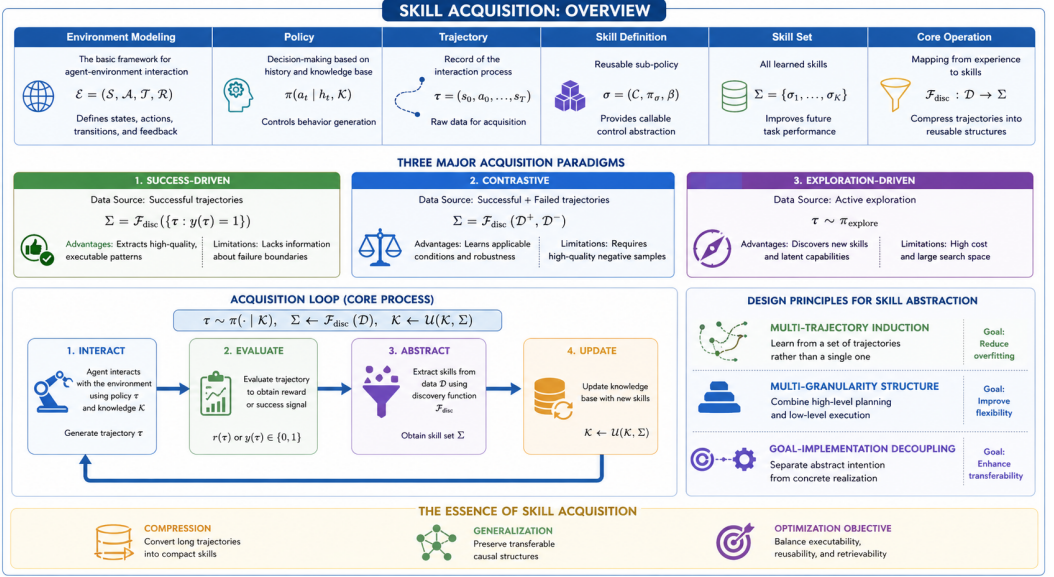


Fig. 11. An overview of agent skill acquisition, illustrating three regimes: (1) success-driven discovery, which abstracts skills from successful trajectories; (2) contrastive discovery, which infers skills from both successful and failed trajectories; and (3) exploration-driven discovery, which actively generates new experience to discover skills. It highlights how acquisition transforms raw interaction into reusable procedural abstractions.

Key Takeaways

The key difficulty is to control the right level of abstraction. If a skill is too specific, it degenerates into a memorized trajectory fragment, leading to reduced reusability; if it is too abstract, it lacks procedural knowledge and becomes difficult to execute or verify.

Recent work addresses this tension through several design principles. First, skills may be induced from a pool of trajectories rather than updated sequentially from individual traces, reducing overfitting to trajectory-local lessons [77]. Second, skill libraries may maintain multiple granularities, separating high-level planning knowledge from fine-grained action or correction knowledge [104, 106]. Third, the abstract goal of a skill can be separated from its concrete implementation, allowing the same procedural intent to be realized differently across environments [141]. These principles indicate that acquisition is not simply a summarization problem. It is an abstraction problem constrained simultaneously by executability, transferability, and future retrievability. In more advanced settings, the iterative loop of acquisition becomes co-evolutionary: the policy determines what experiences are generated, the experiences determine what skills are discovered, and the resulting skill repository in turn shapes future policy behavior [52, 120, 123, 135, 147]. The acquisition stage therefore marks a transition from static prompting or isolated task solving toward continual self-improvement, where agents progressively reorganize experience into reusable competence.

4.2 Storage Stage

The storage stage determines whether a skill library functions as a reusable capability substrate or merely as an ever-growing cache of past artifacts. A flat repository can preserve skills, but it does

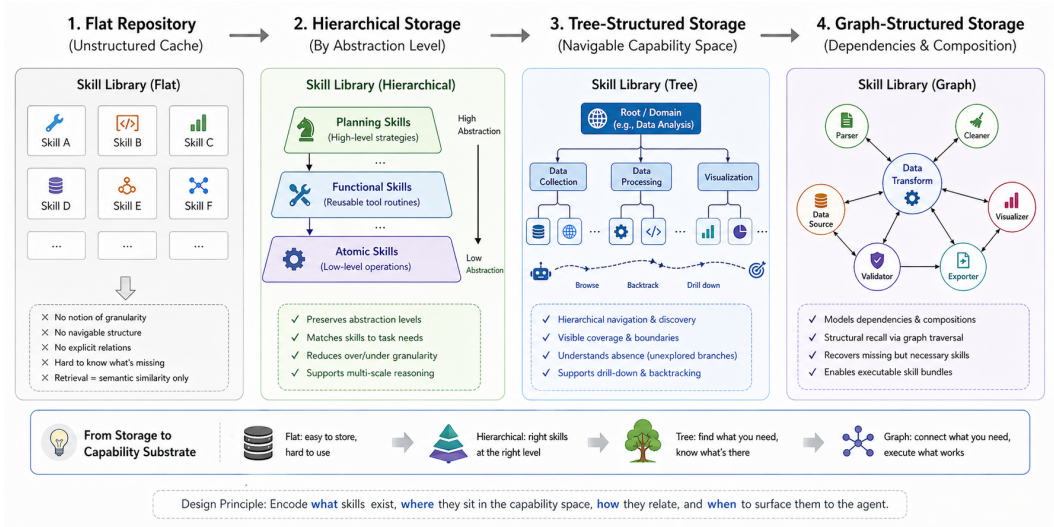


Fig. 12. An overview of agent skill library organization for storage, illustrating four paradigms: (1) unstructured flat storage, (2) hierarchical decomposition by skill granularity, (3) tree-based navigable capability organization, and (4) graph-structured representation capturing dependencies, composition, and relational recall. It illustrates how increasing structure improves discoverability, reuse, and executable coordination of skills.

not expose their granularity, topical locality, prerequisites, compositional affordances, or reliability. The recent literature therefore converges on a central design principle: skill storage must encode not only *what* skills exist, but also *where* they sit in a capability space, *how* they relate to other skills, and *when* they should be surfaced to an agent. In this sense, storage is not a passive persistence layer; it is an organizational interface between formation, usage, and evolution.

Hierarchical storage for skill granularity. A first line of work organizes skills by abstraction level. SkillX explicitly argues that transferable experience should not be stored as monolithic trajectories, but decomposed into planning skills, functional skills, and atomic skills, separating high-level task organization from reusable tool subroutines and low-level execution constraints [106]. SkillRL similarly distills noisy trajectories into a hierarchical SkillBank containing general strategic skills and task-specific heuristics, so that retrieval can trade off broad guidance against local decision support [123]. D2Skill sharpens this view by maintaining task skills for trajectory-level planning and step skills for fine-grained action correction, making storage responsive to different temporal scales in agentic RL [104]. The shared insight is that skill libraries should not be indexed only by semantic similarity to the current task; they should preserve the level of abstraction at which a skill is meant to operate [52]. Without this granularity, agents either overuse broad advice where procedural details are needed, or overload the context with low-level fragments when high-level planning would suffice.

Tree-structured storage for navigable discovery. A second line of work treats the library as a navigable capability space. AgentSkillOS constructs a capability tree by recursively partitioning skills into capability categories, allowing the agent to traverse from coarse domains to specific skills before pruning and ranking candidates [48]. CORPUS2SKILL applies a similar principle to enterprise knowledge: instead of hiding hierarchy behind a vector index, it materializes the corpus

as a skill directory that the agent can browse, backtrack through, and drill into [100]. SkillFoundry extends this idea to scientific domains by organizing heterogeneous resources through a domain knowledge tree, then attaching mined skill packages, verification status, provenance, tests, and metadata to the corresponding regions of the domain structure [95]. Across these systems, the tree is not merely a retrieval accelerator; it gives the agent a map of the library. Compared with flat retrieval, tree-structured storage makes absence and coverage partially observable: the agent can reason about unexplored branches, topic boundaries, and whether evidence or procedural knowledge has been searched sufficiently.

Graph-structured storage for dependency and composition. Tree structures improve discovery, but they impose a mostly single-parent view of skills. For execution, however, skills often have non-tree relations: one skill may depend on another, compose with several alternatives, or substitute for a semantically similar skill. Graph of Skills addresses this by constructing an executable skill graph whose edges encode prerequisite and workflow relations; inference-time retrieval then expands from semantic-lexical seeds through graph diffusion to recover skills that are not topically closest to the query but are necessary for execution [59]. SkillNet generalizes this direction into an ecosystem-scale ontology with three coupled layers: a taxonomy for categories and tags, a relation graph for `similar_to`, `depend_on`, `compose_with`, and `belong_to` relations, and a physical skill package library for deployment [55]. The deeper consensus is that large skill repositories require structural recall, not just semantic recall. A flat index may retrieve the most descriptive skill, but miss parsers, setup utilities, converters, validators, or prerequisite workflows that make the skill executable. Graph storage therefore turns the library from a set of candidate instructions into a space of executable bundles.

4.3 Usage Stage

4.3.1 Retrieval & Disclosure. We formalize retrieval as a budget-constrained selection problem over the skill repository. Given a task context c (which may include task instruction, environment state, and interaction history), a skill library $\mathcal{K}$ containing a set of candidate skills Σ , and a retrieval budget B (e.g., token limit or context capacity), the objective is to select a subset of skills $\Sigma' \subseteq \Sigma$ that maximizes downstream utility:

$$\Sigma^* = \arg \max_{\Sigma' \subseteq \Sigma} \mathcal{U}(\Sigma' \mid c) \quad \text{s.t.} \quad \text{Cost}(\Sigma') \leq B, \quad (11)$$

where $\mathcal{U}(\cdot)$ measures the expected contribution of the selected skills to task success (e.g., improved policy performance, reduced reasoning cost, or higher success likelihood), and $\text{Cost}(\cdot)$ captures the context footprint or retrieval overhead.

Key Takeaways

The implementation of file-system based agent-skill disclosure follows a deliberately asymmetric design: the runtime first exposes only a compact catalog of skill names and descriptions, then loads the full `SKILL.md` only after the task appears to match a skill, and finally allows referenced files, assets, or executable scripts to be accessed on demand.

This progressive disclosure pattern treats the description as the routing interface and the body as the execution interface, thereby making context a staged resource rather than a static prompt budget [7, 84]. In this view, retrieval is not merely semantic search over documents, but a runtime control mechanism that decides which procedural knowledge becomes visible to the model, at which granularity, and under which context budget [64, 98].

Recent research converges on the view that retrieval is no longer a shallow matching step over skill metadata, but a context-sensitive selection layer that must jointly account for task intent, environment state, execution history, and long-term memory. Instead of relying on static descriptions, effective systems increasingly couple retrieval with reranking and parameterized instantiation, treating skills as executable units whose relevance depends on both semantic fit and situational compatibility [15, 135]. At the same time, evidence from large-scale skill pools shows that lightweight metadata is often insufficient for reliable routing, while naive exposure of full skill content introduces severe noise and token inefficiency. This tension has led to a shift toward adaptive disclosure mechanisms that selectively expose high-fidelity signals while compressing or deferring non-essential content [25, 159]. Beyond relevance, a deeper limitation of flat retrieval is its inability to capture functional dependencies between skills. Emerging approaches therefore move toward structure-aware retrieval, where the goal is not to identify isolated matches but to assemble a minimal, execution-complete set of interdependent skills under strict context budgets [59]. Currently, skill retrieval is evolving into an intermediate systems layer that mediates between memory and execution, balancing semantic alignment, structural completeness, and token-efficient disclosure rather than optimizing any single dimension in isolation.

4.3.2 Composition, Orchestration & Execution. The skill execution should not be treated as a flat act of invoking retrieved routines, but as a structured planning problem over reusable capabilities. A skill becomes valuable only when it exposes an interface stable enough to be composed with other skills, while hiding implementation details that vary across websites, tools, or agents. This separation allows higher-level plans to be written over semantic operations rather than brittle action traces, making composition more robust under interface variation and environmental change [141, 157].

A consensus is that orchestration requires explicit intermediate structure between retrieval and execution. The agent must decide not only which skills are relevant, but also how their inputs, outputs, preconditions, costs, and failure modes interact. Graph or pipeline representations make these dependencies operational: they constrain execution order, enable parallelism when subtasks are independent, and localize recovery when one component fails. This explains why structured composition consistently outperforms presenting the same skills as an unordered context or allowing the model to implicitly infer the workflow [48, 124].

Finally, execution quality depends on closing the loop between composition, verification, and adaptive routing. Skills are increasingly treated as executable units that must be validated, monitored, and selectively reused rather than passively stored. Effective systems therefore combine skill libraries with runtime checks, performance estimates, and cost aware routing, allowing agents to choose between alternative implementations or collaborators according to the current state. This shifts the role of skills from static memories to active control abstractions that compress experience, expose reusable structure, and support reliable long horizon behavior [13, 44, 110].

4.4 Maintenance Stage

4.4.1 Refinement & Evolution. Agent skills should not be treated as static procedural artifacts, but as mutable externalized policies whose quality is determined only after repeated execution. Refinement therefore shifts the optimization target from the frozen language model to the skill substrate itself: instructions, triggers, workflow structures, memory routines, code fragments, retrieval policies, and validation rules. This perspective is shared across studies that update skills from failed executions, reward gaps, historical rollouts, or deployment feedback, while leaving model parameters unchanged [62, 75, 162]. The resulting paradigm is a form of non-parametric

Paper	Update Trigger	Optimization Evidence	Evolution Target	Optimization Operator	Validation / Admission	Mechanism Characteristics
SkillTracer [51]	Execution failure (node-level)	Trace-based verification, failure node attribution	Plan-graph structured skills	Local structural repair, parameter refinement	Node-wise executability checks	Structural attribution; localized repair with partial reuse
MCE [139]	Iterative evaluation cycles	Rollout performance, validation metrics, history	CE skills + context artifacts	Agentic crossover, skill recombination	Validation-based selection (best-of-history)	Meta-level evolution; co-evolving skills and context
Skill-Pro [75]	Batch trajectory updates	Rewards, advantages, semantic gradients	Procedural skill pool (Skill-MDP)	Semantic gradient refinement, candidate generation	PPO Gate (trust-region), score pruning	Non-parametric PPO-style evolution with stability constraints
MemSkill [148]	Periodic hard-case mining	Failure cases, reward signals, ground truth	Memory skill bank + selector	Skill refinement and new skill proposal	Rollback, early stopping, RL-based selection	Closed-loop evolution of memory operations as skills
EvoSkill [3]	Task execution failures	Execution traces, prediction vs ground truth	Structured skill folders	Failure-driven proposal and skill construction	Held-out validation, Pareto frontier selection	Failure-driven skill discovery with transferability
Memento-Skills [162]	Interaction feedback loop	Task performance, stateful prompts	Skill memory (externalized skill files)	Read-Write reflective updates, skill rewriting	Retrieval alignment and RL-based routing	Skill-as-memory paradigm; continual external learning
D2Skill [104]	Performance gap (skill vs baseline rollouts)	Paired rollout difference, utility signals	Dual-granularity skill bank (task + step skills)	Reflection-based generation and refinement	Utility-aware pruning and retrieval	RL-coupled skill learning with credit assignment
CoEvoSkills [147]	Verification failure	Surrogate test feedback, pass/fail signals	Multi-file skill packages	Iterative refinement via generator-verifier loop	Surrogate + oracle verification	Co-evolution of skill generator and verifier
SkillClaw [72]	Cross-user interaction patterns	Aggregated trajectories, success/failure evidence	Shared skill repository (SkillHub)	Collective refinement and skill creation	In-the-wild validation	Collective evolution across users and sessions
SkillForge [62]	Deployment failure / low-quality outputs	Expert references, LLM judge, failure diagnosis	Domain-specific skill artifacts	Diagnose-rewrite optimization pipeline	Consistency evaluation, iterative deployment feedback	End-to-end refinement loop with domain grounding

Table 2. An overview of agent skill refinement and evolution. It summarizes how existing frameworks trigger updates, leverage optimization evidence, define evolution targets, apply refinement operators, and enforce validation or admission mechanisms, highlighting diverse design choices for improving reusable skills.

continual learning, where the agent accumulates capability by rewriting the reusable interface through which it acts.

A deeper consensus is that useful refinement requires attribution rather than undifferentiated reflection. Simple post-hoc critique over full trajectories is too coarse to determine whether failure arises from skill selection, activation conditions, missing domain knowledge, incorrect tool usage, brittle grounding, or an invalid internal step. Recent methods therefore introduce increasingly structured evidence channels: node-level execution checks, paired baseline and skill-injected rollouts, hard-case buffers, failure clusters, expert references, and verifier-generated tests [51, 62, 104, 148]. The common principle is to transform raw interaction traces into localized repair signals. Skill evolution becomes effective only when the system can map an observed outcome back to the specific part of the skill that caused it.

Another important pattern is the separation between proposal and admission. Since LLM-generated skill edits can easily overfit individual failures or introduce regressions, most systems decouple mutation from acceptance. Candidate skills may be produced through semantic gradients, crossover, failure diagnosis, collective trajectory aggregation, or generator-verifier interaction, but they are admitted only after passing a gate such as held-out validation, PPO-style trust-region filtering, utility scoring, rollback tests, Pareto selection, or surrogate verification [3, 75, 139, 147]. This suggests that the central technical challenge is not merely generating new skills, but maintaining a stable evolutionary pressure that rewards reusable improvement while suppressing noisy textual drift.

The granularity of evolution is also becoming more explicit. Some work refines internal graph nodes or local action descriptions, preserving valid substructures while repairing only the failing components [51]. Other work evolves entire skill folders, memory skills, context-engineering routines, or dual-granularity skill banks that distinguish task-level guidance from step-level correction [3, 104, 139, 148]. This reflects a broader design principle: skills must support both exploitation and plasticity. Fine-grained edits enable safe local repair, whereas coarse-grained creation enables the discovery of new reusable behaviors when existing abstractions are insufficient.

Skill evolution is most powerful when evidence is accumulated beyond a single episode. Cross-task transfer, multi-user aggregation, historical support tickets, long-term memory, and co-evolving verifiers all enlarge the evidence base from which skill updates are derived [62, 72, 147, 162]. This turns skills into a collective memory layer for agent systems. The optimization objective is no longer to solve the current task alone, but to distill recurring success and failure patterns into reusable procedures that improve future agents.

Key Takeaways

In this sense, refinement and evolution provide the mechanism by which agent skills move from hand-authored prompts toward self-maintaining operational knowledge.

4.4.2 Deprecation & Retirement. A negative maintenance mechanism is required, which removes obsolete procedural knowledge rather than only accumulating new variants. As the skill bank grows, redundancy and stale guidance can degrade retrieval quality, increase selection overhead, and even mislead policy optimization; thus retirement should be understood as an essential counterpart to refinement. Recent studies operationalize this principle through score-based or utility-aware pruning, where skills with non-positive empirical gain, low eviction scores, duplicate semantics, or weak contribution under paired rollouts are gradually phased out from the active pool [75, 104, 111]. This makes deprecation a performance-grounded decision rather than a purely administrative label: a skill is not retired because it is old, but because its expected marginal value under the current agent, task distribution, and competing skill alternatives has fallen below the cost of keeping it retrievable.

At the system level, however, retirement should still be staged rather than abrupt. Deprecated skills should first be marked in metadata and documentation, excluded from new adoption, monitored for remaining dependencies, mapped to preferred alternatives, and only then removed from active deployment while preserving historical versions and usage logs for auditability [76]. In this sense, robust skill lifecycle management combines two complementary logics: algorithmic pressure that continuously demotes low-utility skills, and governance pressure that preserves traceability, migration safety, and backward compatibility during final retirement.

4.5 Internalization Stage

Internalization marks a qualitative transition from externally managed skills to intrinsic behavioral priors of the agent. Rather than continuously expanding skill repositories, recent work converges on the view that mature systems should amortize reliable procedural knowledge into the policy itself, enabling frequently used strategies to be executed without retrieval or prompt-level scaffolding [67, 108, 119]. In this sense, internalization serves as the closure of the lifecycle, transforming accumulated experience into parameter-level competence.

Formally, internalization can be viewed as a conditional-to-unconditional policy compression process. Let $\pi(a \mid h, \sigma)$ denote a skill-conditioned policy that relies on an external skill $\sigma \in \Sigma$, and let $\pi_\theta(a \mid h)$ denote the base policy without explicit skill access. Internalization seeks to transfer the behavior induced by σ into the parameters θ by minimizing a divergence between the two:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{h \sim \mathcal{D}, \sigma \sim \Sigma} \left[D(\pi(\cdot \mid h, \sigma), \pi_\theta(\cdot \mid h)) \right], \quad (12)$$

where $D(\cdot \mid \cdot)$ may be instantiated as KL divergence or other behavior-matching objectives. This formulation captures internalization as a form of skill-conditioned distillation, where externally scaffolded behavior is amortized into the base policy.

This stage resolves a core tension in skill-augmented agents. While external skills offer modularity and reusability, persistent reliance on them incurs retrieval noise, context overhead, and a form of behavioral dependence where the model follows instructions without fully acquiring them [67, 108]. Internalization reframes skill repositories as training scaffolds rather than permanent dependencies. Across diverse paradigms—including experience distillation, hint-guided reinforcement learning, meta-experience learning, and skill-conditioned self-distillation—a shared principle emerges: skills should function as structured learning signals that reshape the policy, rather than remain transient context augmentations [37, 108, 119, 149].

Crucially, internalization must be selective. Not all useful skills warrant absorption, as some are redundant, overly specific, or context-dependent. Accordingly, existing approaches incorporate validation mechanisms prior to consolidation. Utility-aware pruning filters consistently beneficial skills [104], while contrastive analyses between successful and failed trajectories extract generalizable meta-experience suitable for parameterization [37]. Curriculum-based schemes further regulate the transition by gradually removing explicit skill guidance as the policy becomes self-sufficient [67], and self-distillation transfers skill-conditioned behavior into a student that operates without such support [108]. Collectively, these methods position internalization as a controlled compression process, selectively translating structured external knowledge into stable parametric behavior, while leaving volatile or infrequent knowledge in external form.

5 Layer 4: Runtime Integration

Runtime integration is the layer where abstract capability becomes executable behavior. In the following subsections, Section 5.1 develops the interface layer through the terminal and tool interfaces, while Section 5.2 turns to system-level coordination through multi-agent and harness integration. Figure 13 summarizes how these runtime components fit together around skills as the mediating layer.

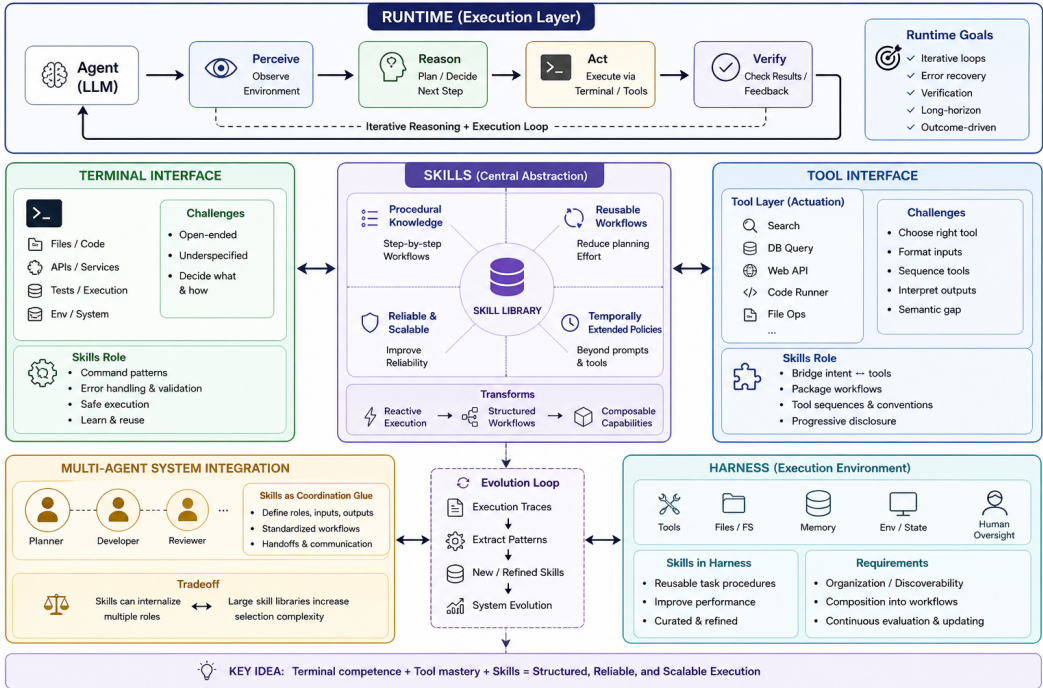


Fig. 13. An overview of the runtime integration of agent skills across interfaces, system integration, and harness-level execution.

5.1 Interface Accessing

5.1.1 Terminal Interface. The terminal interface provides a distinctive runtime substrate because it collapses perception, action, and verification into a unified text-based execution loop. The command-line environment exposes a compact and programmable surface: commands can inspect files, query APIs, transform data, install dependencies, execute tests, and modify persistent state. This makes the terminal not merely another tool endpoint, but a general-purpose operating layer through which agents can perform long-horizon technical work. Recent evaluations show that realistic terminal tasks require agents to explore partially observed environments, chain interdependent commands, recover from execution errors, and satisfy outcome-based tests rather than reproduce a prescribed trajectory [74]. In enterprise settings, this low-level interface can even match or outperform more elaborate GUI- or schema-mediated architectures, because direct programmatic access allows the agent to discover APIs, compose operations flexibly, cache intermediate artifacts, and recover from malformed requests or missing abstractions [9]. Thus, the terminal should be viewed as a runtime integration interface in which the agent's reasoning loop is grounded by executable feedback and persistent environmental state.

Skills become especially important in this setting because the terminal is powerful precisely where it is underspecified: it offers an open action space, but does not itself determine which command sequences are safe, reusable, or semantically appropriate for a task. A skill layer can constrain this open-endedness by packaging recurring command patterns, diagnostic procedures, API conventions, verification routines, and human-review checkpoints into reusable procedural units. From the runtime perspective, such skills mediate between high-level intent and shell-level execution: they guide the agent in selecting commands, structuring intermediate files, interpreting errors, and deciding when an operation is ready for verification.

Work on terminal-oriented task synthesis further suggests that terminal competence is best understood as traversal over scenarios and skills, where each step applies an appropriate capability to transform the environment toward the goal [23]. Continual skill-learning systems extend this view by reflecting on past trajectories and converting successful or corrected behavior into reusable skill files, enabling agents to improve across terminal sessions rather than rely only on transient context [47]. At the same time, because terminal agents often execute consequential operations, skill-mediated review layers can expose plans, code changes, memory updates, and command effects to humans in structured interfaces before irreversible actions occur [165]. Consequently, terminal runtime integration should not be reduced to shell access alone; it is a layered architecture in which skills organize executable behavior, persistence, feedback, and oversight around the command-line environment.

5.1.2 Tool Interface. Tool interfaces are not an optional extension to the agent runtime, but a constitutive boundary through which an otherwise text-bound model becomes an operational system. A runtime must decide when external interaction is necessary, bind an intention to a callable interface, supply well-formed arguments, observe the returned state, and incorporate that observation into subsequent reasoning. Early work already framed tool use as a learned control problem: the model must determine not only which external API is relevant, but also when to invoke it, what input to pass, and how the result should condition later prediction [92]. More recent protocol-oriented work further shows that real-world agent execution requires more than isolated function calls: the runtime must support standardized discovery, invocation, feedback, and multi-step coordination across heterogeneous services [29]. In this sense, the tool interface is the runtime's actuation layer, translating linguistic plans into externally grounded operations.

However, exposing tools directly to the agent creates a semantic gap between high-level task intent and low-level executable affordances. Tool schemas are precise but narrow: they specify inputs, outputs, and execution constraints, yet they do not by themselves encode when a tool should be used, how multiple tools should be sequenced, which intermediate results matter, or what domain-specific conventions should govern the workflow. As the number of tools grows, this gap becomes a runtime scalability problem: the agent must repeatedly perform tool discovery, parameter binding, dependency tracking, error recovery, and result synthesis. Thus, the runtime cannot be understood merely as a loop over atomic tool calls; it requires an intermediate layer that organizes tool affordances into reusable, task-relevant procedures. Without such a layer, tool use remains reactive and instance-level, forcing the agent to reconstruct the same orchestration logic from scratch across structurally similar tasks [29, 66].

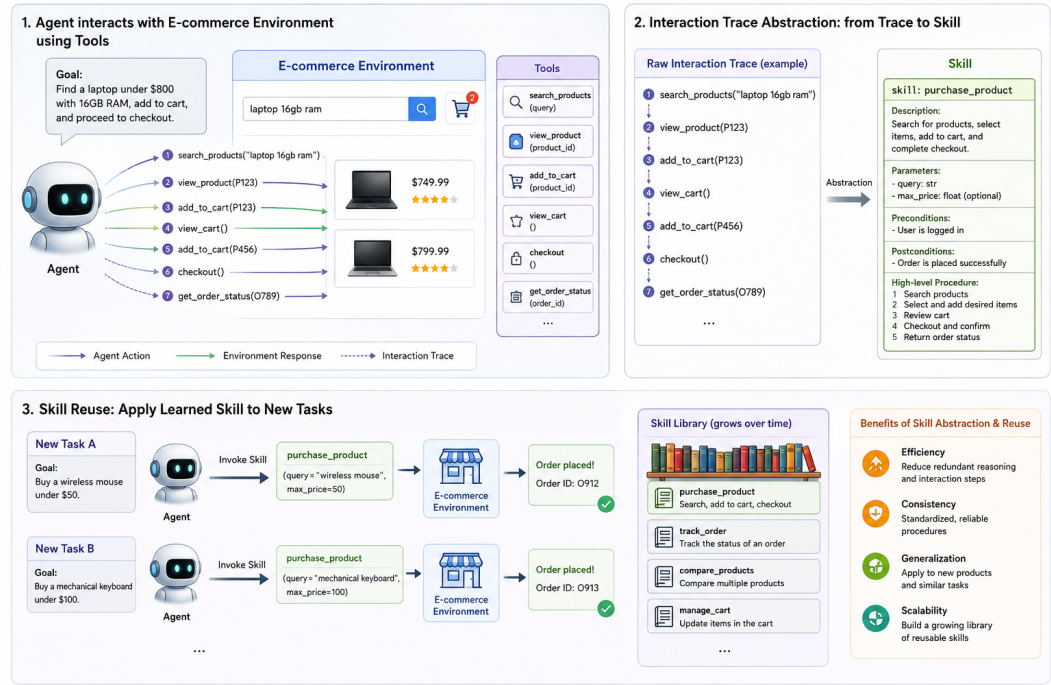


Fig. 14. A case study of agent skill as the intermediate layer between high-level intent and low-level tool interfaces in e-commerce environment. The skill encodes when to use which tools and how to sequence them while the runtime executes the skill's procedural logic and manages the tool interactions.

Key Takeaways

Skills occupy this intermediate layer between runtime deliberation and tool-interface execution. **Rather than treating a skill as an isolated artifact in a lifecycle, runtime integration views it as a contextual and procedural adapter:** it packages task-relevant instructions, scripts, references, and tool-use patterns so that the agent can specialize its behavior only when the current situation calls for it.

This design supports progressive disclosure: lightweight descriptors guide skill selection, while detailed instructions and auxiliary resources are loaded only when needed, reducing context pressure while preserving access to deep procedural knowledge [4, 132]. Formally, a skill can be interpreted as a temporally extended policy with a descriptor and a reference mechanism: the runtime first selects or reveals the appropriate context, then executes an intra-skill workflow that may call tools, run code, inspect files, or coordinate external services [132]. The skill layer therefore turns the tool interface from a flat action set into a structured repertoire of situated capabilities.

Procedural knowledge improves agentic workflows because it externalizes task structure: it decomposes abstract goals into ordered subtasks, tracks progress, preserves dependencies, and verifies completion before moving forward [35]. Similarly, compositional tool-use studies show that long-horizon tasks often contain recurring substructures, where successful sequences of atomic tool calls can be abstracted, verified, cached, and reused as higher-level executable skills [13]. From a runtime perspective, this means that skills are neither merely prompts nor merely tools. They are a

mediation layer that amortizes planning cost, constrains tool orchestration, carries domain-specific execution norms, and exposes a more stable interface between the agent's reasoning loop and the heterogeneous tools it must operate.

5.2 System Integration

5.2.1 Multi-Agent System Integration. Multi-agent system integration turns the runtime from a single execution loop into an organizational substrate in which reasoning, delegation, verification, and communication must be coordinated across multiple specialized actors. The main benefit of this architecture is not simply that more model instances are available, but that complex work can be decomposed into role-specific responsibilities, intermediate artifacts, and handoff protocols. In such settings, skills function as the procedural glue that stabilizes collaboration: they encode what each role is responsible for, what form its output should take, how it should consume messages from other agents, and when its result is ready to be passed onward. This reduces the ambiguity of open-ended dialogue and transforms inter-agent communication into a more disciplined workflow of structured artifacts, subscriptions, and executable feedback [33].

At the same time, skills also expose a trade-off between distributed coordination and internalized capability. A multi-agent system can be interpreted as a set of specialized behavioral policies connected by a communication graph, while a skill-based agent can internalize some of these policies as selectable procedures. This suggests that skills can sometimes replace explicit agent multiplicity by compressing roles into a reusable skill library, thereby reducing communication cost, latency, and redundant context exchange. However, this substitution is not universal: as the skill library grows, selection itself becomes a runtime bottleneck, especially when skills are semantically similar. Multi-agent integration therefore benefits from hierarchical skill organization, where coarse routing first narrows the behavioral region and finer selection then activates the appropriate procedure [49].

More generally, skills make multi-agent integration adaptive rather than static. They can serve not only as role instructions, but also as inspectable design knowledge about which topology, routing rule, interface contract, or specialist role is appropriate for a class of tasks. Execution traces and failure cases can be converted into revised skills, new specialist templates, or updated coordination rules, allowing the runtime to evolve its organization without changing model parameters. This is especially important when task demands vary across modalities, domains, or decomposition structures: the communication graph should depend on both the current input and the active capabilities of the agents. In this view, skills mediate a feedback loop between agent expertise and system topology, enabling the runtime to refine how agents collaborate as their reusable capabilities improve [78, 97, 162].

5.2.2 Harness Integration. Agent harnesses occupy an ecological position between foundation models and concrete runtime environments. Recent agent systems make this position visible: terminal-first assistants, editor-centered coding agents, cloud-sandbox software agents, and open-source local assistants all provide not merely model access, but runtime envelopes in which commands, files, tools, memory, and human review are coordinated [6, 17, 27, 79, 81, 83]. In this sense, the harness is the practical substrate through which an agent becomes executable: it determines what the agent can observe, which actions it may take, how actions are constrained, and how success or failure is recorded [39].

Within this layer, agent skills serve as an intermediate procedural substrate. A skill packages reusable task knowledge, including activation conditions, workflow steps, domain conventions, scripts, resources, and verification expectations, so that the harness does not expose the agent only to a flat space of tools and files. Empirical work suggests that curated skills can substantially

improve task completion, while automatically generated skills remain fragile unless they are evaluated, revised, and reused through structured feedback loops [50, 160]. Other studies further show that skill libraries become more useful when they are organized for discovery, composed into dependency-aware workflows, and refined from execution trajectories rather than stored as isolated textual advice [48, 152, 153].

Key Takeaways

Consequently, the design goal of future harnesses is not only to provide more tools, but to embed a skill-aware mediation layer into the runtime itself.

Such a harness should retrieve relevant skills from lightweight descriptors, load detailed procedural content only when needed, bind skills to tools and workspace state, monitor whether the expected procedure is followed, and update the skill library from verified successes and diagnosed failures. It should also expose skill invocation, command effects, generated artifacts, and verification results to humans when tasks are consequential. In this architecture, skills transform the harness from a generic execution shell into a learning runtime: the harness supplies observation, actuation, persistence, and safety, while the skill layer supplies reusable procedural structure that can be tested, composed, and improved over time [48, 82, 145].

6 Layer 5: Governance

This section introduces the governance layer of agent skills. As illustrated in Figure 15, governance concerns how skill-based agent systems should decide which skills to trust, restrict, or avoid when reusable skills are created, shared, installed, and executed. We first examine the emerging marketplaces and ecosystems that distribute skills at scale in Section 6.1. We then analyze the threat and risk surface introduced by reusable skills in Section 6.2, including prompt-injection amplification, compositional vulnerabilities, and capability misuse. Finally, Section 6.3 surveys safe usage and governance mechanisms, such as access control, sandboxing, monitoring, human oversight, and policy compliance, that aim to make skill execution more trustworthy and auditable.

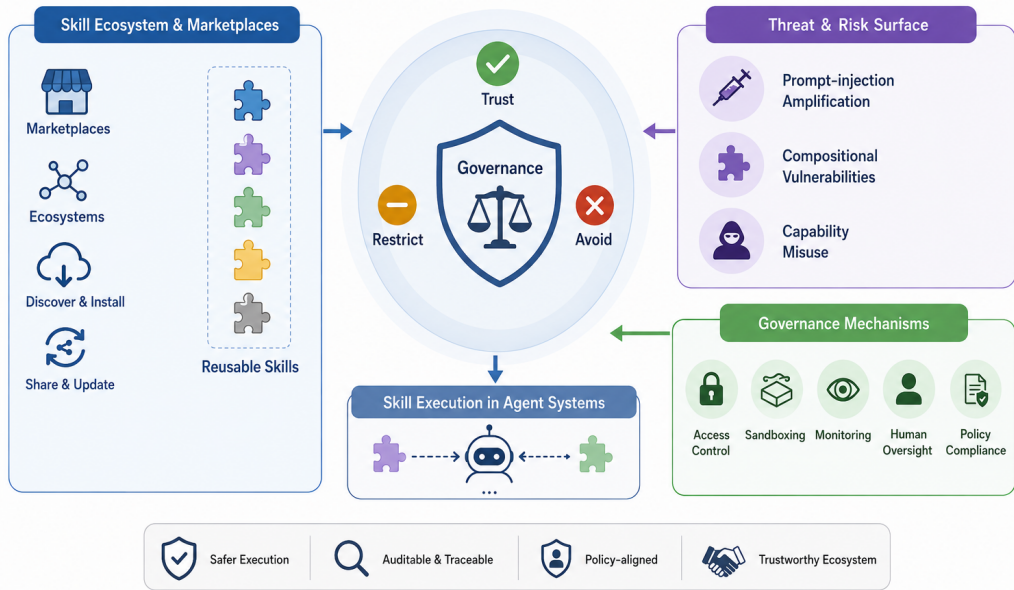


Fig. 15. An overview of the governance of agent skills, which encompasses the marketplaces and ecosystems where skills are shared, the threat and risk surface introduced by skills, and the mechanisms for safe usage and governance.

6.1 Marketplaces and Ecosystems

As the adoption of agent skills accelerates, an ecosystem of *skill marketplaces* and reusable repositories has rapidly emerged. Similar to software package registries such as PyPI or npm, these marketplaces support the discovery, sharing, installation, and reuse of modular agent capabilities. Recent measurements show that the ecosystem has expanded extremely quickly, with public marketplaces hosting tens of thousands of skills within only a few months [56]. By 2026, multiple marketplaces had already appeared, including open community registries, GitHub-indexed repositories, curated commercial catalogs, and official vendor-maintained directories [89]. This rapid growth reflects increasing demand for reusable and composable agent functionality across software engineering, productivity, retrieval, and content-generation workflows.

Marketplace	Skills listed	Curation	Security review	Creator payments	MCP live access
Skills.sh ¹	~2,000	Community submissions	None	No	No
SkillsMP ²	800,000+	Scraped from GitHub, 2+ stars	None	No	No
LobeHub ³	169,000+	Scraped	None	No	No
ClaudeSkills.info ⁴	658	Community + official	None	No	No
MCP Market ⁵	~500	Submission-based	None	No	Partial
Anthropic official ⁶	~20	Manual, Anthropic-verified	Internal audit	No	No
Claude Marketplaces ⁷	2,500+ registries	Per-owner	None	No	No
Agensi ⁸	200+	Manual + automated	8-point scan on every submission	Yes, 80/20 split	Yes, \$9/mo Pro tier

Table 3. Comparison of AI Skill and MCP Marketplaces [89]. There are eight marketplaces worth knowing about in April 2026. Each is built on a different assumption about what developers want.

Different marketplaces adopt substantially different governance models. Open repositories such as `skills.sh` and SkillsMP prioritize scale and openness, allowing community-driven publication with minimal barriers, while curated platforms emphasize security review, manual verification, and creator accountability [89]. At the same time, skills are increasingly treated as reusable digital assets that can be versioned, ranked, monetized, and integrated across heterogeneous agent frameworks. Recent industry analyses even describe skills as a new form of software supply chain, where third-party behavioral modules become critical infrastructure for extending agent capabilities [46]. However, empirical studies also reveal high redundancy and fragmentation in current ecosystems, with many marketplaces containing large numbers of duplicated or near-identical skills [56].

Key Takeaways

Despite their rapid growth, current skill ecosystems still face substantial challenges in security, privacy, and governance. Unlike traditional prompts, skills often interact with local environments, external APIs, package managers, and sensitive user data, which significantly expands the attack surface.

Recent investigations have identified malicious or vulnerable skills capable of remote command execution, credential leakage, dependency hijacking, and covert supply-chain manipulation [1, 46].

¹<https://skills.sh/>

²<https://skillsmp.com/>

³<https://lobehub.com/skills>

⁴<https://claudeskills.info/>

⁵<https://mcpmarket.com/>

⁶<https://claude.com/skills>

⁷<https://claudemarketplaces.com/>

⁸<https://www.agensi.io/>

These findings suggest that skill marketplaces inherit many of the risks of traditional software ecosystems while introducing new threats specific to agentic systems.

6.2 Threat and Risk Surface

Agent skills act as *persistent carriers of prompt injection*, shifting attacks from transient inputs to reusable system components. Adversarial instructions can be embedded within skill definitions, auxiliary scripts, or even hidden documentation layers, allowing them to bypass inspection while remaining visible to the model [93, 113]. Recent works further show that such attacks can be automated and optimized to be stealthy and context-aware, achieving high success rates in realistic agent settings [40]. Empirical evaluations and systematization studies reveal that current agents are highly vulnerable to these skill-based injections, often executing harmful actions such as data exfiltration or unauthorized tool use, and that existing defenses remain insufficient against adaptive attacks [73, 94]. Skills transform prompt injection into a modular and persistent threat, where the attack surface shifts from user inputs to the skills themselves.

Beyond injection-based threats, skills themselves may be *intrinsically malicious or compromised*, introducing risks at the level of skill implementation and supply chain. Backdoor attacks can be embedded either in bundled models or in skill logic, enabling hidden payloads that activate only under specific triggers while maintaining benign behavior [24, 103]. At a larger scale, empirical studies reveal that a non-trivial portion of real-world skills are explicitly harmful or malicious, capable of executing actions such as data exfiltration, fraud, or agent hijacking, often operating with user-level privileges and minimal vetting [43, 63]. These findings highlight that skills introduce a *software supply-chain risk*, where seemingly benign and reusable components may conceal adversarial behaviors that are difficult to detect through standard evaluation. Crucially, such risks do not remain isolated to individual skills but can be exposed through the broader ecosystem in which skills are shared and reused.

Furthermore, security risks emerge at the level of the *skill ecosystem*, where large-scale sharing, reuse, and propagation amplify vulnerabilities. Empirical studies show that public skill marketplaces contain widespread security issues, with a significant fraction of skills exhibiting vulnerabilities such as data exfiltration, privilege escalation, or malicious intent [36, 65, 125]. The ecosystem structure further exacerbates these risks: skills are highly redundant and frequently cloned, allowing vulnerabilities to silently propagate across large numbers of derivatives without visibility [164]. Moreover, the open and loosely regulated nature of skill distribution introduces classic software supply-chain threats, including large-scale injection of malicious artifacts and lack of provenance or verification guarantees [10, 86].

To investigate the above risks, recent works focus on *detection, auditing, and red teaming frameworks* for systematically identifying vulnerable or malicious skills. Multi-stage auditing systems leverage static analysis, LLM-based reasoning, and multi-agent collaboration to detect semantic inconsistencies and simulate combinatorial risks across skill interactions [28, 34, 70]. Complementarily, red-teaming approaches dynamically probe skills through adversarial prompting, demonstrating that even benign skills can be exploited under realistic interactions [21]. Large-scale empirical analyses further highlight the challenges of accurate detection, showing that naive scanners may significantly overestimate risk without contextual signals such as repository provenance, while also uncovering new attack vectors in the ecosystem [32]. Finally, system-level studies emphasize that effective defense requires holistic security frameworks rather than isolated point solutions [19]. These works suggest that governing agent skills demands not only identifying malicious artifacts, but also continuously auditing their behavior and interactions within complex agent systems [146].

6.3 Safe Usage and Governance Mechanisms

In this subsection, we organize existing governance efforts into three complementary perspectives: (1) access control and trust management for selective admission, (2) runtime guardrails for enforcing behavioral constraints during execution, and (3) formal governance mechanisms for providing system-level investigation. Together, these explore frameworks that make skill execution remain within safe, predictable, and auditable bounds, while preserving the flexibility and scalability of skill-based agent systems.

Access Control and Trust Management. A fundamental governance mechanism for agent skills is *access control and trust management*, which determines whether a skill should be admitted into the system in the first place. In practice, skills are often distributed through open marketplaces or external repositories, making provenance and trustworthiness critical concerns. Practical guidelines emphasize installing skills only from trusted sources and performing manual auditing of instructions, dependencies, and external connections prior to deployment, highlighting the importance of pre-execution vetting as a first line of defense¹. Systematic analyses further reinforce this need by identifying risks and advocating for trust-tiered execution models, where skills are granted different levels of privilege based on their origin and verification status [42]. Recent empirical evidence also demonstrates that insufficient vetting can lead to severe credential leakage risks in third-party skills, including hardcoded secrets, prompt-injection-induced disclosure, and exploitable logging behaviors [16]. Together, these approaches frame governance as a *selective admission problem*, where controlling what enters the skill ecosystem is essential to mitigating downstream risks.

Runtime Guardrails and Policy Enforcement. Beyond admission control, governance must actively constrain the *procedural execution of skills* through runtime guardrails. While traditional frameworks such as Guardrails² and NeMo Guardrails were initially designed to filter LLM dialogue, they are increasingly adapted to govern skills by treating each skill as a black-box function with enforceable invariants [88]. In this context, guardrails act as an orchestration shim that intercepts the input arguments and tool outputs of a skill to ensure adherence to safety policies. Unlike model-level alignment, which is often bypassed during complex tool-use, these programmable rules operate independently of the underlying model to restrict risky behaviors. This paradigm shifts the role of guardrails from simple content moderation to *behavioral mediation*, treating even verified skills as partially untrusted components. By applying these constraints at the skill boundary, developers can enforce structured execution and inter-step policy validation, ensuring that multi-step skill sequences do not deviate from the intended system safety logic during runtime.

Formal Governance and System-Level Control. Finally, a more principled approach to governance is achieved through *formal and system-level control mechanisms*, which provide explicit, enforceable guarantees over skill behavior. Agent Contracts exemplify this direction by defining formal specifications that integrate input/output constraints, resource limits, temporal bounds, and success criteria into a unified governance framework, enabling predictable and auditable execution across complex workflows [140]. Furthermore, systemic governance involves organizing these capabilities within structured ontologies and standardized evaluation dimensions. By categorizing skills based on their safety requirements, executability, and cost-awareness, infrastructure systems can support more consistent management across diverse agentic platforms [55]. These approaches elevate governance from heuristic filtering to *principled system design*, where skills are treated as governed entities with well-defined constraints and evaluation criteria, enabling scalable and trustworthy deployment in real-world agent systems [53, 109].

¹<https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>

²<https://github.com/guardrails-ai/guardrails>

7 Layer 6: Application

In this section, we survey how agent skills support downstream applications across embodied robotics and physical manipulation (Section 7.1), open-ended game environments (Section 7.2), web agents and browser automation (Section 7.3), GUI, mobile, and OS agents (Section 7.4), and software engineering and coding agents (Section 7.5). As shown in Figure 16, reusable procedural knowledge strengthens agent interaction across these settings by bridging high-level intent and environment-specific execution.

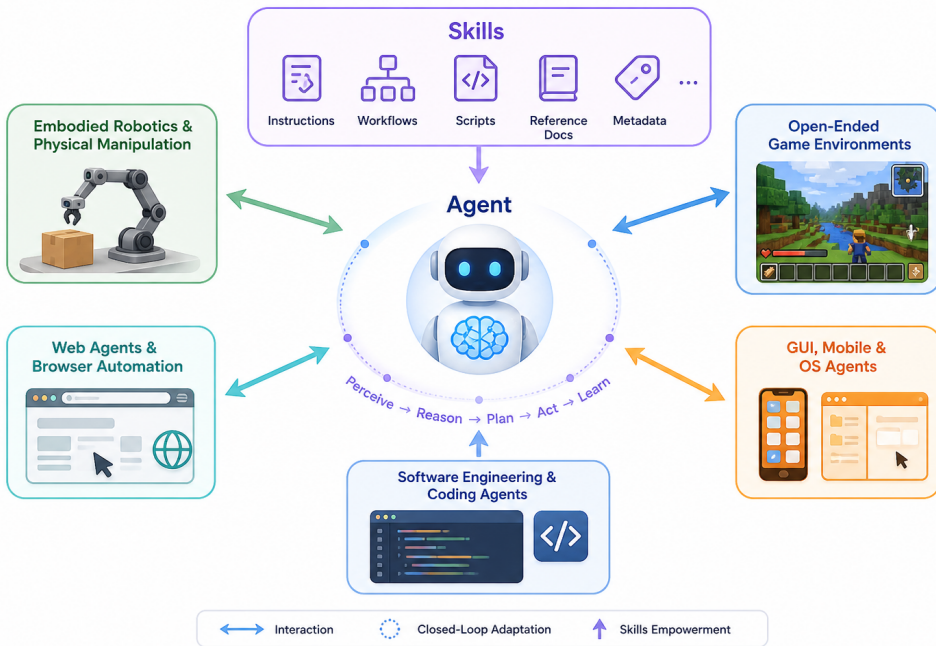


Fig. 16. An overview of agent skills as reusable procedural knowledge that strengthens an agent’s interaction with diverse environments, including robotics, games, web automation, GUI/OS control, and software engineering.

7.1 Embodied Robotics & Physical Manipulation

In embodied robotics and physical manipulation, skills serve as an intermediate contract between open-ended semantic reasoning and physically constrained control. Early work shows that language models become practically useful for robots only when their high-level proposals are filtered through embodiment-specific affordances: a candidate action is not merely selected because it is linguistically plausible, but because a corresponding low-level controller is likely to succeed in the current state [2]. This view reframes a skill as both an executable behavior and a grounding primitive: its natural-language description exposes it to symbolic reasoning, while its value or feasibility estimate anchors it in the robot’s perceptual and dynamical reality. Subsequent work extends this contract across embodiments by discovering reusable manipulation prototypes from unaligned human and robot videos, then transferring these prototypes into robot action policies and composing them for unseen tasks [130]. The deeper implication is that skills are not only action

primitives; they are also a representational currency through which demonstrations, language, and control can be made mutually legible.

A second line of work shifts the field from fixed skill repertoires toward evolving skill ecosystems [126]. Rather than assuming that the robot begins with a complete library, recent systems use foundation models to propose meaningful manipulation tasks, synthesize reward and success conditions, launch reinforcement learning, and verify learned behaviors with independent multimodal evaluators [155]. Related lifelong approaches treat past executions as a memory substrate from which new compositional skills can be abstracted, added to a callable library, and reused in later long-horizon tasks without directly modifying the underlying model parameters [105]. Multimodal memory-based agents further suggest that experience should not be stored as raw trajectories alone, but distilled into reusable abstractions about causal relations, object-state transitions, temporal subgoals, and task-relevant visual cues [91]. In simulation-heavy pipelines, this trend is amplified by automated scene generation, task generation, atomic skill reuse, and feedback-driven training optimization, reducing the cost of collecting embodied trajectories for new tasks [118]. Together, these works reveal a conceptual transition: skill is becoming a writable, retrievable, and inspectable form of embodied knowledge, rather than a static controller hidden beneath a planner.

The most recent studies emphasize that scalable skill growth must be paired with verification, runtime governance, and domain-specific procedural structure. Real-time process verification addresses the fact that embodied skill acquisition is expensive to evaluate exhaustively; by scoring scene configurations, subtask decompositions, training supervisions, and success metrics before or during learning, verification models can improve both sample efficiency and robustness [143]. In contact-rich industrial manipulation, explicit skill libraries complement end-to-end vision-language-action policies by imposing stage-specific constraints, enabling failure detection and recovery, and stabilizing long-horizon operations that require precise physical contact [58]. Finally, capability-centric formulations argue that long-lived embodied agents should preserve a stable cognitive identity while evolving modular, versioned capabilities under a safety-governed runtime layer [85]. This perspective suggests that the central challenge is no longer simply how to acquire more skills, but how to maintain a trustworthy skill lifecycle: discovering, composing, verifying, updating, and retiring embodied capabilities without eroding safety, interpretability, or behavioral consistency.

7.2 Open-Ended Game Environments

Open-ended game environments provide a natural testbed for studying agent skills because they combine sparse rewards, partial observability, long-horizon dependency, and an effectively unbounded task space. In such settings, skills are not merely low-level policies, but reusable abstractions that make open-endedness tractable: they compress repeated interaction patterns into units that can be retrieved, composed, evaluated, and transferred [96, 102, 142]. Early Minecraft-based work highlights this transition by coupling large-scale game knowledge with thousands of language-specified tasks, allowing agents to learn from human-generated demonstrations, tutorials, and discussions rather than from isolated reward engineering alone [22]. More recent evaluation frameworks further emphasize that open-ended game agents should be assessed not only by success on fixed tasks, but also by their ability to generalize across diverse initial states, compose atomic tasks into harder objectives, and preserve task quality under automatic generation [158]. From the perspective of agent skills, these environments expose a central tension: an agent must acquire stable behavioral routines, yet the value of each routine depends on changing context, available resources, and downstream goals.

A major line of work therefore treats skills as an external and expandable memory for lifelong game playing. Instead of solving every task from scratch, agents progressively store executable

behaviors, retrieve relevant ones, and refine them through feedback from the environment, execution errors, or self-verification [107]. This makes skill acquisition cumulative: simple actions such as gathering resources, crafting tools, or fighting enemies become prerequisites for increasingly complex achievements, and the resulting library supports transfer to new worlds or tasks [61]. Multimodal extensions make this abstraction more grounded by using visual perception to interpret the environment, language reasoning to decompose high-level goals, and skill-code retrieval to transform decomposed instructions into executable actions [156]. A complementary trend moves away from human-designed workflows and studies how agents can evolve skills through trial-and-reasoning from raw visual interaction, using implicit signals such as visual change or game progression rather than explicit APIs or rewards [20]. These studies suggest that skills serve as the interface between perception, reasoning, and control: they are where unstructured experience becomes reusable procedural knowledge.

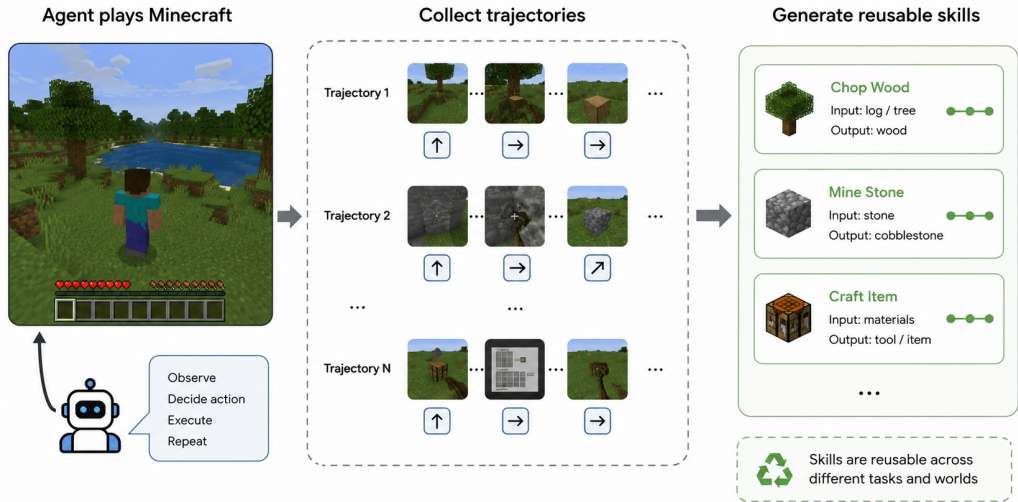


Fig. 17. A case of agent skills in open-ended game environments. The agent learns reusable behavioral patterns that guide future interaction across diverse tasks and changing contexts.

Recent work further deepens the role of skills by making their organization and grounding explicit. In strategy games and other GUI-based environments, isolated skill memories can lead to myopic reuse; structuring experience as a graph allows visually different but functionally related states to share strategic value, so that skills become edges in a broader map of long-term decision making [101]. In parallel, hierarchical formulations combine language-based symbolic planning with reinforcement learning, treating skills as operators with preconditions and effects while using execution trajectories to correct inaccurate language priors [144]. Another direction couples the decision policy and the skill bank itself, allowing the agent to discover, segment, refine, and reuse skills from unlabeled rollouts while simultaneously improving its ability to select them during long-horizon play [120]. Taken together, open-ended games show that agent skill is evolving from a static library of useful routines into a dynamic substrate for continual learning: skills encode what the agent has learned, constrain what it can plan, and determine how efficiently it can turn future experience into new competence.

7.3 Web Agents & Browser Automation

In web agents and browser automation, skills emerge as a crucial abstraction for converting low-level, brittle interaction traces into reusable procedural knowledge. Early work on web agents revealed that realistic browser environments are characterized by long horizons, noisy observations, dynamic page states, and multiple valid solution paths, making simple imitation of reference trajectories insufficient for robust automation [18, 31]. In this setting, a skill can be viewed as an intermediate unit between primitive actions, such as clicking or typing, and high-level user intents, such as comparing products, retrieving account information, or completing a multi-page form. This abstraction is particularly valuable because web tasks often contain repeated subroutines across websites and domains: search, filter, navigate, verify, submit, and recover from unexpected page states [163]. Thus, skill-based agents do not merely memorize demonstrations; they attempt to distill trajectories into reusable interaction patterns that reduce planning depth and improve generalization.

A central trend in this line of work is the gradual shift from textual procedural memory toward more executable and verifiable forms of skills. Some studies show that agents can extract reusable workflows from prior trajectories and retrieve them to guide future decisions, improving performance on long-horizon web benchmarks while reducing unnecessary steps [117]. However, natural-language workflows remain weakly grounded: they may describe what should be done without guaranteeing that the corresponding browser actions are valid in the current page state. Later work therefore represents skills as parameterized programs, APIs, or executable action sequences, allowing agents to compose primitive browser operations into higher-level routines and verify whether the induced procedure actually succeeds [116, 157]. This transition is important: it reframes skill learning from passive memory retrieval into an active process of abstraction, execution, testing, debugging, and reuse. In browser automation, the most useful skills are therefore not only semantically meaningful but also operationally grounded.

Reinforcement-learning-based approaches indicate that long-horizon web tasks benefit from fine-grained feedback that distinguishes efficient, valid trajectories from merely successful but redundant ones, and from mechanisms that consolidate high-quality trajectories into reusable and composable skill structures [133]. Other work emphasizes that web skills should expose both executable structure and interpretable step-level guidance, so that agents can either invoke a skill directly for efficiency or follow it adaptively when the environment changes [115]. A complementary perspective treats skills as explicit procedural objects with preconditions, postconditions, recovery rules, and local repair operations, which improves reliability when self-generated skills fail under realistic multimodal web interaction [68]. Taken together, these studies show that skills are not merely prompts or cached demonstrations; they are becoming modular, inspectable, and transferable control interfaces for web agents. The broader implication is that scalable browser automation will likely depend less on one-shot reasoning over raw webpages and more on continuously maintained skill libraries that accumulate procedural experience, support composition across tasks, and remain repairable as websites evolve.

7.4 GUI, Mobile & OS Agents

GUI, mobile, and operating-system agents instantiate a particularly demanding setting for agent skills, because the action space is not a stable API but a heterogeneous surface of pixels, accessibility trees, keyboard/mouse events, application states, and sometimes scripts. Early work on general computer-use agents shows that a universal interface can make agents applicable across terminals, files, web pages, and desktop applications, but also exposes a scalability problem: without reusable skills, the agent must repeatedly rediscover how to open applications, navigate menus, fill forms,

manipulate documents, or recover from UI errors [80, 122]. Interactive benchmarks for mobile and desktop operating systems further emphasize that real performance depends on outcome-level success under changing states, parameterized tasks, and multi-application workflows, rather than imitation of a single demonstration trajectory [11, 87]. In this sense, skills are not merely auxiliary tools; they become an intermediate operating layer that maps high-level user intents into grounded, verifiable procedures over GUIs.

Recent skill-centric systems converge on three design patterns. First, agents can accumulate procedural tools or action routines through self-directed exploration, turning successful executions into reusable capabilities for unseen applications [122]. Second, trajectories can be abstracted hierarchically: low-level executions encode how to perform operations, mid-level routines encode what subgoal they realize, and higher-level schemas encode relations among subgoals; this structure is especially useful for long-horizon mobile and web workflows where many small UI decisions must be coordinated [127]. Third, desktop and professional-software settings benefit from parameterized skills with execution and composition graphs, or from environment exploration that discovers unit functions and composes them into expert workflows; these approaches reduce repeated planning, support argument instantiation, and improve reliability on tasks requiring fine-grained manipulation [15, 60]. The common insight is that GUI skills should carry both semantic intent and executable grounding: a skill must describe when it applies, which UI entities it expects, how parameters bind to visible state, and how success or failure is detected.

The mobile setting makes this shift even more salient because repeated personal tasks often recur with small linguistic or parameter changes while the UI remains structurally similar. Compiling successful mobile trajectories into parameterized action templates allows later invocations to be replayed with far fewer model calls, converting a stateless per-step reasoner into a system whose reliability can improve through use [12]. Across these works, skill-based GUI agents thus move the application paradigm from “reason every step” toward “learn, validate, retrieve, compose, and maintain procedures”. This also reveals open challenges: skills must remain robust to app updates, screen-size variation, permissions, pop-ups, and visual elements absent from accessibility trees; they must compose across apps without propagating hidden state errors; and their execution should be inspectable and interruptible for safety. Therefore, in GUI/mobile/OS agents, the central research question is not only how to acquire skills, but how to manage their lifecycle as living procedural knowledge in dynamic digital environments [11, 12, 87, 127].

7.5 Software Engineering & Coding Agents

Software engineering has become a particularly compelling application domain for agent skills because repository-level development exposes a gap between conventional code generation and the practices required to resolve real issues. Unlike self-contained programming tasks, real-world issue resolution requires agents to localize relevant files in large codebases, reason over cross-file dependencies, edit code without introducing regressions, construct or interpret tests, and decide when a patch is sufficient under execution-based feedback [14, 45]. In this setting, skills should not be viewed merely as prompt snippets, but as reusable procedural abstractions that organize long-horizon development behavior. Recent work suggests that such abstractions can be induced through structured training on localization, repair, test construction, and self-reflection, thereby providing priors that make subsequent multi-turn agent adaptation more efficient and less brittle [136]. A related line of work further shows that many apparently different coding tasks share intermediate competencies, such as repository exploration, implementation, and reasoning; training these components through scalable auxiliary environments can improve generalization beyond the specific downstream task on which the agent is evaluated [128]. Thus, the central value of skills in coding agents is compositional: they factor complex software work into learnable, reusable

units that can be recombined across bug fixing, testing, library construction, refactoring, and other development workflows.

A second strand of evidence refines this view by treating skills as explicit objects to be evaluated, selected, and optimized. Requirement-driven evaluation shows that skill injection is not uniformly beneficial: many skills provide no measurable pass-rate improvement, while some even degrade performance when their guidance conflicts with the target repository or version context [30]. This finding is important because it challenges the assumption that adding more procedural knowledge is always helpful. In software engineering, a skill competes for context budget and may bias the agent toward inappropriate conventions; its utility therefore depends on domain fit, abstraction level, and compatibility with the concrete project state. More specialized evidence comes from efficiency optimization, where reusable slow-to-fast transformation patterns can be mined from program pairs and represented as portable optimization skills. Such skills shift optimization from instance-level rewriting toward mechanism-level reuse, enabling agents to diagnose likely bottlenecks, retrieve relevant transformation strategies, compose plans, and generate optimized candidates even without runtime feedback [114]. This suggests that skills are most valuable when they encode recurring mechanisms rather than surface examples.

Taken together, these studies indicate an emerging design principle for coding agents: scalable software autonomy should be built around compact, evaluable, and composable skills rather than around monolithic task success alone. One approach formalizes core capabilities such as code localization, code editing, unit-test generation, issue reproduction, and code review as atomic units with structured outputs and sandbox-grounded rewards; joint optimization over these units improves both the individual skills and transfer to unseen composite tasks [71]. Another complementary direction optimizes skill bundles themselves under multiple objectives, showing that pruning and substitution can improve pass rate and reduce cost more effectively than simply accumulating additional instructions [26]. These results point to a mature view of agent skills for software engineering: skills are not static prompt assets, but trainable, measurable, and optimizable interfaces between model capability and engineering practice.

8 Challenges and Future Opportunities

In this section, we highlight several cross-cutting challenges that are not confined to any single layer of the skill stack, and outline future opportunities for building skill-based agents as reliable, adaptive, and governable systems.

8.1 Measuring Skill Value Beyond Task Success

The current evaluation remains too dependent on downstream task success. While task success is important, it does not reveal why a skill helps, whether it transfers, or whether its benefit comes from the skill content, the retrieval mechanism, the runtime environment, or the underlying model. A skill may increase success by providing genuine procedural knowledge, but it may also succeed by leaking benchmark-specific heuristics, overfitting to a narrow environment, or biasing the model toward a lucky action pattern. Conversely, a useful skill may not immediately improve success rate but may reduce planning cost, improve interpretability, or increase recovery ability after failure.

Future benchmarks should therefore evaluate agent skills as first-class objects rather than as hidden components inside agent pipelines. This requires decomposed metrics: activation accuracy, execution fidelity, transfer across tasks, robustness under environmental change, marginal utility over the base agent, interaction with other skills, and negative interference when irrelevant skills are retrieved. A mature evaluation protocol should also separate three questions: whether the skill is well written, whether the agent retrieves it at the right time, and whether the runtime can execute it safely. Without this decomposition, empirical results will remain difficult to interpret.

8.2 The Skill–Context Trade-off

Skills are often introduced as a way to reduce reasoning burden, but they also compete for the agent’s limited context and attention. A large skill library can improve coverage while simultaneously increasing retrieval noise. A detailed skill can reduce ambiguity while consuming more context. A concise skill can be cheap to load but may omit preconditions, exceptions, or verification steps. This creates a fundamental trade-off between expressiveness and runtime efficiency.

Future work should treat skill usage as an information allocation problem. Instead of asking only which skill is relevant, systems should ask how much of the skill should be disclosed, at what time, and in what form. Progressive disclosure is one promising design, but it remains under-theorized. Future agents may need adaptive skill compression, task-conditioned views of the same skill, and runtime mechanisms that expand or collapse skill details according to uncertainty, risk, and execution progress. This points toward a broader research direction: context should not be seen merely as a token budget, but as a dynamic control surface for procedural knowledge.

8.3 Skill Interference and Behavioral Conflict

As skill repositories grow, agents will increasingly face situations where multiple skills are relevant but mutually inconsistent. One skill may recommend conservative verification before acting, while another prioritizes efficiency. One may encode domain conventions for a specific software stack, while another encodes a conflicting convention from a different project. In multi-agent or marketplace settings, skills authored by different people may carry incompatible assumptions, hidden priorities, or duplicated procedures.

This raises the problem of skill interference. Existing research often assumes that adding useful skills monotonically improves capability, but in practice skill libraries may create behavioral conflicts. Future systems need mechanisms for detecting, ranking, and resolving conflicts among skills. This may require typed skill interfaces, explicit preconditions, priority rules, provenance-aware selection, and contradiction checks before execution. More ambitiously, agents may need a

skill arbitration layer that reasons not only about relevance, but also about compatibility with the current task, user intent, environment state, and other active skills.

8.4 The Boundary Between Adaptation and Drift

Skill evolution enables agents to improve over time, but it also introduces the risk of procedural drift. A skill that is repeatedly rewritten from local failures may gradually lose its original scope, accumulate exceptions, or become optimized for recent cases at the expense of generality. In collaborative settings, different users may modify the same skill toward different preferences, creating forks that are difficult to reconcile. In safety-critical contexts, automatic refinement may silently change the agent's behavior in ways that users do not notice.

Future research should study skill evolution as a stability problem, not only as an optimization problem. This requires version-aware evaluation, regression testing, change summaries, rollback mechanisms, and long-term monitoring of skill behavior. More importantly, systems should distinguish between local patches and conceptual updates. A local patch fixes a narrow failure case; a conceptual update changes the underlying procedure. Treating both as ordinary "skill refinement" obscures their different risks. A promising direction is to develop skill governance workflows analogous to software engineering practices: issue tracking, semantic versioning, review gates, test suites, and release channels for procedural knowledge.

8.5 Skills as Infrastructure, Not Add-ons

A final challenge is architectural. Many current systems treat skills as optional add-ons to an otherwise complete agent: retrieve a skill, insert it into context, and let the model proceed. This design is simple, but it underestimates the infrastructural role skills may play. If skills are to support long-horizon, high-stakes, or multi-user agents, they must be integrated with identity, permissions, storage, versioning, testing, monitoring, and audit systems.

The future of agent skills may therefore resemble the evolution of software packages. Early skills are easy to write and share, but large-scale deployment will require registries, dependency management, compatibility checks, permission models, provenance tracking, and runtime isolation. This shift changes the research question. The central problem is no longer only how to make an individual skill useful, but how to build an ecosystem in which many skills can coexist, evolve, compose, and remain trustworthy over time.

Overall, the next stage of agent-skill research should move from artifact design to ecosystem design. The field has already demonstrated that skills can improve agent behavior in many domains. The more difficult question is how to make skills scientifically defined, empirically measurable, context-efficient, conflict-aware, evolution-stable, human-governable, and infrastructurally reliable. Addressing these problems will determine whether skills remain a useful engineering trick or become a foundational layer for the next generation of adaptive agent systems.

9 Conclusion

In this paper, we have presented a comprehensive survey of agent skills as a key mechanism for enabling LLM-based agents to transform experience, human expertise, and external resources into reusable procedural knowledge. We argue that skills occupy an intermediate position between memory and rules: they are more general than individual episodic traces, yet more operational than abstract principles. By compressing repeated interactions into structured procedures, skills help agents reduce planning burden, stabilize long-horizon behavior, and improve their ability to act across similar tasks and environments.

To organize this rapidly growing area, we proposed a six-layer taxonomy covering ontology, representation and packaging, lifecycle, runtime integration, governance, and applications. This taxonomy shows that agent skills should not be understood merely as prompt fragments or isolated tool-use recipes. Instead, they form an evolving capability substrate that can be represented as natural-language guidance, executable code, decision graphs, filesystem-based packages, or structured records; acquired from trajectories, external resources, and human expertise; stored and retrieved through hierarchical or graph-based libraries; and integrated into terminal interfaces, tool interfaces, multi-agent systems, and agent harnesses.

At the same time, the development of agent skills introduces important challenges. As skill libraries grow, systems must address issues of retrieval accuracy, composition, lifecycle maintenance, evaluation, and selective internalization. Moreover, because skills may access tools, files, APIs, and sensitive user data, they also expand the security and governance surface of agent systems. Trust management, provenance tracking, sandboxing, auditing, runtime guardrails, and formal execution constraints will therefore be essential for ensuring that skills remain reliable, controllable, and safe in real-world deployment.

Looking forward, agent skills are likely to become a central component of practical agent infrastructure. Future research should move beyond simply collecting more skills toward building skill ecosystems that are discoverable, composable, verifiable, and self-improving. By treating skills as governed and evolving procedural artifacts, future LLM agents can become more capable not only because their underlying models improve, but also because their externalized knowledge, execution environments, and reusable capabilities become better organized. In this sense, agent skills provide an important path toward more reliable, adaptive, and trustworthy agent systems.

References

- [1] David Abutbul. 2026. When Your Plugin Starts Picking Your Dependencies: Marketplace Skills and Dependency Hijack in Claude Code. Prompt Security Blog. <https://prompt.security/blog/when-your-plugin-starts-picking-your-dependencies-marketplace-skills-and-dependency-hijack-in-claude-code> Accessed: 2026-05-06.
- [2] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. arXiv:2204.01691 [cs.RO] <https://arxiv.org/abs/2204.01691>
- [3] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. 2026. EvoSkill: Automated Skill Discovery for Multi-Agent Systems. arXiv:2603.02766 [cs.AI] <https://arxiv.org/abs/2603.02766>
- [4] Anthropic. 2025. Equipping agents for the real world with Agent Skills. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills> Accessed: 2026-05-07.
- [5] Anthropic. 2026. Agent Skills. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview> Claude Docs. Accessed: 2026-05-07.
- [6] Anthropic. 2026. Claude Code by Anthropic: AI Coding Agent, Terminal, IDE. <https://www.anthropic.com/claude-code> Accessed: 2026-05-05.
- [7] Anthropic. 2026. *Skill Authoring Best Practices*. Claude API Docs. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices>
- [8] Anthropic. 2026. What are Skills? Claude Help Center. <https://support.claude.com/en/articles/12512176-what-are-skills> Accessed 2026-04-26.
- [9] Patrice Bechard, Orlando Marquez Ayala, Emily Chen, Jordan Skelton, Sagar Davasam, Srinivas Sunkara, Vikas Yadav, and Sai Rajeswar. 2026. Terminal Agents Suffice for Enterprise Automation. arXiv:2604.00073 [cs.SE] <https://arxiv.org/abs/2604.00073>
- [10] Varun Pratap Bhardwaj. 2026. SkillFortify: Formal Analysis and Supply Chain Security for Agentic AI Skills. doi:10.5281/ZENODO.18787663
- [11] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Buckner, Lawrence Jang, and Zack Hui. 2024. Windows Agent Arena: Evaluating Multi-Modal OS Agents at Scale. arXiv:2409.08264 [cs.AI] <https://arxiv.org/abs/2409.08264>
- [12] Qijia Chen, Andrea Bellucci, Zhida Sun, and Giulio Jacucci. 2026. SkillDroid: Compile Once, Reuse Forever. arXiv:2604.14872 [cs.HC] <https://arxiv.org/abs/2604.14872>
- [13] Shiqi Chen, Jingze Gai, Ruochen Zhou, Jinghan Zhang, Tongyao Zhu, Junlong Li, Kangrui Wang, Zihan Wang, Zhengyu Chen, Klara Kaleb, Ning Miao, Siyang Gao, Cong Lu, Manling Li, Junxian He, and Yee Whye Teh. 2026. SkillCraft: Can LLM Agents Learn to Use Tools Skillfully? arXiv:2603.00718 [cs.CL] <https://arxiv.org/abs/2603.00718>
- [14] Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. 2026. SWE-Exp: Experience-Driven Software Issue Resolution. arXiv:2507.23361 [cs.SE] <https://arxiv.org/abs/2507.23361>
- [15] Tianyi Chen, Yinheng Li, Michael Solodko, Sen Wang, Nan Jiang, Tingyuan Cui, Junheng Hao, Jongwoo Ko, Sara Abdali, Leon Xu, Suzhen Zheng, Hao Fan, Pashmina Cameron, Justin Wagle, and Kazuhito Koishida. 2026. CUA-Skill: Develop Skills for Computer Using Agent. arXiv:2601.21123 [cs.AI] <https://arxiv.org/abs/2601.21123>
- [16] Zhihao Chen, Ying Zhang, Yi Liu, Gelei Deng, Yuekang Li, Yanjun Zhang, Jianting Ning, Leo Yu Zhang, Lei Ma, and Zhiqiang Li. 2026. Credential Leakage in LLM Agent Skills: A Large-Scale Empirical Study. arXiv:2604.03070 [cs.CR] <https://arxiv.org/abs/2604.03070>
- [17] Cursor. 2026. Cursor: The best way to code with AI. <https://cursor.com/> Accessed: 2026-05-05.
- [18] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. arXiv:2306.06070 [cs.CL] <https://arxiv.org/abs/2306.06070>
- [19] Xinhao Deng, Yixiang Zhang, Jiaqing Wu, Jiaqi Bai, Sibo Yi, Zhuoheng Zou, Yue Xiao, Rennai Qiu, Jianshan Ma, Jialuo Chen, Xiaohu Du, Xiaofang Yang, Shiwen Cui, Changhua Meng, Weiqiang Wang, Jiaxing Song, Ke Xu, and Qi Li. 2026. Taming OpenClaw: Security Analysis and Mitigation of Autonomous LLM Agent Threats. arXiv:2603.11619 [cs.CR] <https://arxiv.org/abs/2603.11619>
- [20] Jiawei Du, Jinlong Wu, Yuzheng Chen, Yucheng Hu, Bing Li, and Joey Tianyi Zhou. 2025. Rethinking Agent Design: From Top-Down Workflows to Bottom-Up Skill Evolution. arXiv:2505.17673 [cs.AI] <https://arxiv.org/abs/2505.17673>
- [21] Zenghao Duan, Yuxin Tian, Zhiyi Yin, Liang Pang, Jingcheng Deng, Zihao Wei, Shicheng Xu, Yuyao Ge, and Xueqi Cheng. 2026. SkillAttack: Automated Red Teaming of Agent Skills through Attack Path Refinement. arXiv:2604.04989 [cs.CR] <https://arxiv.org/abs/2604.04989>

- [22] Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. 2022. MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge. arXiv:2206.08853 [cs.LG] <https://arxiv.org/abs/2206.08853>
- [23] Zhiyuan Fan, Tinghao Yu, Yuanjun Cai, Jiangtao Guan, Yun Yang, Dingxin Hu, Jiang Zhou, Xing Wu, Zhuo Han, Feng Zhang, and Lilin Wang. 2026. Toward Scalable Terminal Task Synthesis via Skill Graphs. arXiv:2604.25727 [cs.AI] <https://arxiv.org/abs/2604.25727>
- [24] Yunhao Feng, Yifan Ding, Yingshui Tan, Boren Zheng, Yanming Guo, Xiaolong Li, Kun Zhai, Yishan Li, and Wenke Huang. 2026. SkillTrojan: Backdoor Attacks on Skill-Based Agent Systems. arXiv:2604.06811 [cs.CR] <https://arxiv.org/abs/2604.06811>
- [25] Yudong Gao, Zongjie Li, Yuanyuanyuan, Zimo Ji, Pingchuan Ma, and Shuai Wang. 2026. SkillReducer: Optimizing LLM Agent Skills for Token Efficiency. arXiv:2603.29919 [cs.SE] <https://arxiv.org/abs/2603.29919>
- [26] Jingzhi Gong, Ruizhen Gu, Zhiwei Fei, Yazhuo Cao, Lukas Twist, Alina Geiger, Shuo Han, Dominik Sobania, Federica Sarro, and Jie M. Zhang. 2026. SkillMOO: Multi-Objective Optimization of Agent Skills for Software Engineering. arXiv:2604.09297 [cs.SE] <https://arxiv.org/abs/2604.09297>
- [27] Google. 2026. Gemini CLI. <https://google-gemini.github.io/gemini-cli/> Accessed: 2026-05-05.
- [28] Zihan Guo, Zhiyu Chen, Xiaohang Nie, Jianghao Lin, Yuanjian Zhou, and Weinan Zhang. 2026. SkillProbe: Security Auditing for Emerging Agent Skill Marketplaces via Multi-Agent Collaboration. arXiv:2603.21019 [cs.CR] <https://arxiv.org/abs/2603.21019>
- [29] Zikang Guo, Benfeng Xu, Chiwei Zhu, Wentao Hong, Xiaorui Wang, and Zhendong Mao. 2025. MCP-AgentBench: Evaluating Real-World Language Agent Performance with MCP-Mediated Tools. arXiv:2509.09734 [cs.CL] <https://arxiv.org/abs/2509.09734>
- [30] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? arXiv:2603.15401 [cs.SE] <https://arxiv.org/abs/2603.15401>
- [31] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. arXiv:2401.13919 [cs.CL] <https://arxiv.org/abs/2401.13919>
- [32] Florian Holzbauer, David Schmidt, Gabriel Gegenhuber, Sebastian Schrittwieser, and Johanna Ullrich. 2026. Malicious Or Not: Adding Repository Context to Agent Skill Classification. arXiv:2603.16572 [cs.CR] <https://arxiv.org/abs/2603.16572>
- [33] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352 [cs.AI] <https://arxiv.org/abs/2308.00352>
- [34] Yinghan Hou and Zongyou Yang. 2026. SkillSieve: A Hierarchical Triage Framework for Detecting Malicious AI Agent Skills. arXiv:2604.06550 [cs.CR] <https://arxiv.org/abs/2604.06550>
- [35] Vincent Hsiao, Mark Roberts, and Leslie Smith. 2025. Procedural Knowledge Improves Agentic LLM Workflows. arXiv:2511.07568 [cs.AI] <https://arxiv.org/abs/2511.07568>
- [36] Haichuan Hu, Ye Shang, and Quanjun Zhang. 2026. Red Skills or Blue Skills? A Dive Into Skills Published on ClawHub. arXiv:2604.13064 [cs.CL] <https://arxiv.org/abs/2604.13064>
- [37] Shiting Huang, Zecheng Li, Yu Zeng, Qingnan Ren, Zhen Fang, Qisheng Su, Kou Shi, Lin Chen, Zehui Chen, and Feng Zhao. 2026. Internalizing Meta-Experience into Memory for Guided Reinforcement Learning in Large Language Models. arXiv:2602.10224 [cs.LG] <https://arxiv.org/abs/2602.10224>
- [38] Xu Huang, Junwu Chen, Yuxing Fei, Zhuohan Li, Philippe Schwaller, and Gerbrand Ceder. 2026. CASCADE: Cumulative Agentic Skill Creation through Autonomous Development and Evolution. arXiv:2512.23880 [cs.AI] <https://arxiv.org/abs/2512.23880>
- [39] Haonian Ji, Kaiwen Xiong, Siwei Han, Peng Xia, Shi Qiu, Yiyang Zhou, Jiaqi Liu, Jinlong Li, Bingzhou Li, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. ClawArena: Benchmarking AI Agents in Evolving Information Environments. arXiv:2604.04202 [cs.LG] <https://arxiv.org/abs/2604.04202>
- [40] Xiaojun Jia, Jie Liao, Simeng Qin, Jindong Gu, Wenqi Ren, Xiaochun Cao, Yang Liu, and Philip Torr. 2026. SkillJect: Automating Stealthy Skill-Based Prompt Injection for Coding Agents with Trace-Driven Closed-Loop Refinement. arXiv:2602.14211 [cs.CR] <https://arxiv.org/abs/2602.14211>
- [41] Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. 2026. XSkill: Continual Learning from Experience and Skills in Multimodal Agents. arXiv:2603.12056 [cs.AI] <https://arxiv.org/abs/2603.12056>
- [42] Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. 2026. SoK: Agentic Skills – Beyond Tool Use in LLM Agents. arXiv:2602.20867 [cs.CR] <https://arxiv.org/abs/2602.20867>

- [43] Yukun Jiang, Yage Zhang, Michael Backes, Xinyue Shen, and Yang Zhang. 2026. HarmfulSkillBench: How Do Harmful Skills Weaponize Your Agents? arXiv:2604.15415 [cs.CR] <https://arxiv.org/abs/2604.15415>
- [44] Zhengbo Jiao, Shaobo Wang, Zifan Zhang, Xuan Ren, Wei Wang, Bing Zhao, Hu Wei, and Linfeng Zhang. 2026. Agentic Proposing: Enhancing Large Language Model Reasoning via Compositional Skill Synthesis. arXiv:2602.03279 [cs.AI] <https://arxiv.org/abs/2602.03279>
- [45] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770 [cs.CL] <https://arxiv.org/abs/2310.06770>
- [46] Claire Kahn, Erich Stuntebeck, and Ravish Chawla. 2026. Skills are the new supply chain. The marketplace is not ready. Airia Blog. <https://airia.com/skills-are-the-new-supply-chain-the-marketplace-is-not-ready/> Accessed: 2026-05-06.
- [47] Letta. 2025. Skill Learning: Bringing Continual Learning to CLI Agents. <https://www.letta.com/blog/skill-learning> Accessed: 2026-05-05.
- [48] Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. 2026. Organizing, Orchestrating, and Benchmarking Agent Skills at Ecosystem Scale. arXiv:2603.02176 [cs.CL] <https://arxiv.org/abs/2603.02176>
- [49] Xiaoxiao Li. 2026. When Single-Agent with Skills Replace Multi-Agent Systems and When They Fail. arXiv:2601.04748 [cs.AI] <https://arxiv.org/abs/2601.04748>
- [50] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Bin Xu, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han chung Lee. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. arXiv:2602.12670 [cs.AI] <https://arxiv.org/abs/2602.12670>
- [51] Yuyang Li, Yiran Dou, Jie-Jing Shao, Yueming Lyu, Ivor Tsang, and Haiyan Yin. 2026. SkillTracer: Structural Failure Attribution and Refinement of Agentic Skills in Long-Horizon Web Tasks. <https://openreview.net/forum?id=OiyEjThGeZ>
- [52] Yu Li, Rui Miao, Zhengling Qi, and Tian Lan. 2026. ARISE: Agent Reasoning with Intrinsic Skill Evolution in Hierarchical Reinforcement Learning. arXiv:2603.16060 [cs.AI] <https://arxiv.org/abs/2603.16060>
- [53] Zhiyuan Li, Jingzheng Wu, Xiang Ling, Xing Cui, and Tianyue Luo. 2026. Towards Secure Agent Skills: Architecture, Threat Taxonomy, and Security Analysis. arXiv:2604.02837 [cs.CR] <https://arxiv.org/abs/2604.02837>
- [54] Qiliang Liang, Hansi Wang, Zhong Liang, and Yang Liu. 2026. From Skill Text to Skill Structure: The Scheduling-Structural-Logical Representation for Agent Skills. arXiv:2604.24026 [cs.CL] <https://arxiv.org/abs/2604.24026>
- [55] Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Xin Xu, Tongtong Wu, Kun Wang, Yang Liu, Zhen Bi, Jungang Lou, Yuchen Eleanor Jiang, Hangcheng Zhu, Gang Yu, Haiwen Hong, Longtao Huang, Hui Xue, Chenxi Wang, Yijun Wang, Zifei Shan, Xi Chen, Zhaopeng Tu, Feiyu Xiong, Xin Xie, Peng Zhang, Zhengke Gui, Lei Liang, Jun Zhou, Chiyu Wu, Jin Shang, Yu Gong, Junyu Lin, Changliang Xu, Hongjie Deng, Wen Zhang, Keyan Ding, Qiang Zhang, Fei Huang, Ningyu Zhang, Jeff Z. Pan, Guilin Qi, Haofen Wang, and Huajun Chen. 2026. SkillNet: Create, Evaluate, and Connect AI Skills. arXiv:2603.04448 [cs.AI] <https://arxiv.org/abs/2603.04448>
- [56] George Ling, Shanshan Zhong, and Richard Huang. 2026. Agent Skills: A Data-Driven Analysis of Claude Skills for Extending Large Language Model Functionality. arXiv:2602.08004 [cs.SE] <https://arxiv.org/abs/2602.08004>
- [57] Anthony Zhe Liu, Jongwook Choi, Sungryull Sohn, Yao Fu, Jaekyeom Kim, Dong-Ki Kim, Xinhe Wang, Jaewon Yoo, and Honglak Lee. 2024. SkillAct: Using Skill Abstractions Improves LLM Agents. <https://openreview.net/forum?id=6LG3clRrF4>
- [58] Chang Liu, Sibao Tian, Xiao Liang, and Minghui Zheng. 2026. SELF-VLA: A Skill Enhanced Agentic Vision-Language-Action Framework for Contact-Rich Disassembly. arXiv:2603.11080 [cs.RO] <https://arxiv.org/abs/2603.11080>
- [59] Dawei Liu, Zongxia Li, Hongyang Du, Xiyang Wu, Shihang Gui, Yongbei Kuang, and Lichao Sun. 2026. Graph of Skills: Dependency-Aware Structural Retrieval for Massive Agent Skills. arXiv:2604.05333 [cs.AI] <https://arxiv.org/abs/2604.05333>
- [60] Jiateng Liu, Zhenhailong Wang, Rushi Wang, Bingxuan Li, Jeonghwan Kim, Aditi Tiwari, Pengfei Yu, Denghui Zhang, and Heng Ji. 2026. OSExpert: Computer-Use Agents Learning Professional Skills via Exploration. arXiv:2603.07978 [cs.AI] <https://arxiv.org/abs/2603.07978>
- [61] Shunyu Liu, Yaoru Li, Kongcheng Zhang, Zhenyu Cui, Wenkai Fang, Yuxuan Zheng, Tongya Zheng, and Mingli Song. 2025. Odyssey: Empowering Minecraft Agents with Open-World Skills. arXiv:2407.15325 [cs.AI] <https://arxiv.org/abs/2407.15325>

- [62] Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. 2026. SkillForge: Forging Domain-Specific, Self-Evolving Agent Skills in Cloud Technical Support. arXiv:2604.08618 [cs.IR] <https://arxiv.org/abs/2604.08618>
- [63] Yi Liu, Zhihao Chen, Yanjun Zhang, Gelei Deng, Yuekang Li, Jianting Ning, Ying Zhang, and Leo Yu Zhang. 2026. Malicious Agent Skills in the Wild: A Large-Scale Security Empirical Study. arXiv:2602.06547 [cs.CR] <https://arxiv.org/abs/2602.06547>
- [64] Yujian Liu, Jiabao Ji, Li An, Tommi Jaakkola, Yang Zhang, and Shiyu Chang. 2026. How Well Do Agentic Skills Work in the Wild: Benchmarking LLM Skill Usage in Realistic Settings. arXiv:2604.04323 [cs.CL] <https://arxiv.org/abs/2604.04323>
- [65] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026. Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale. arXiv:2601.10338 [cs.CR] <https://arxiv.org/abs/2601.10338>
- [66] LlamaIndex. 2026. Skills vs MCP tools for agents: when to use what. <https://www.llamaindex.ai/blog/skills-vs-mcp-tools-for-agents-when-to-use-what> Accessed: 2026-05-05.
- [67] Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. 2026. SKILL0: In-Context Agentic Reinforcement Learning for Skill Internalization. arXiv:2604.02268 [cs.LG] <https://arxiv.org/abs/2604.02268>
- [68] Zijian Lu, Yiping Zuo, Yupeng Nie, Xin He, Weibei Fan, Lianyong Qi, and Shi Jin. 2026. ContractSkill: Repairable Contract-Based Skills for Multimodal Web Agents. arXiv:2603.20340 [cs.SE] <https://arxiv.org/abs/2603.20340>
- [69] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. 2025. Large Language Model Agent: A Survey on Methodology, Applications and Challenges. arXiv:2503.21460 [cs.CL] <https://arxiv.org/abs/2503.21460>
- [70] Lijia Lv, Xuehai Tang, Jie Wen, Jizhong Han, and Songlin Hu. 2026. Structured Security Auditing and Robustness Enhancement for Untrusted Agent Skills. arXiv:2604.25109 [cs.CR] <https://arxiv.org/abs/2604.25109>
- [71] Yingwei Ma, Yue Liu, Xinlong Yang, Yanhao Li, Kelin Fu, Yibo Miao, Yuchong Xie, Zhexu Wang, and Shing-Chi Cheung. 2026. Scaling Coding Agents via Atomic Skills. arXiv:2604.05013 [cs.SE] <https://arxiv.org/abs/2604.05013>
- [72] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. 2026. SkillClaw: Let Skills Evolve Collectively with Agentic Evolver. arXiv:2604.08377 [cs.AI] <https://arxiv.org/abs/2604.08377>
- [73] Narek Maloyan and Dmitry Namiot. 2026. Prompt Injection Attacks on Agentic Coding Assistants: A Systematic Analysis of Vulnerabilities in Skills, Tools, and Protocol Ecosystems. arXiv:2601.17548 [cs.CR] <https://arxiv.org/abs/2601.17548>
- [74] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. arXiv:2601.11868 [cs.SE] <https://arxiv.org/abs/2601.11868>
- [75] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. 2026. Skill-Pro: Learning Reusable Skills from Experience via Non-Parametric PPO for LLM Agents. arXiv:2602.01869 [cs.AI] <https://arxiv.org/abs/2602.01869>
- [76] Milvus. 2026. *How do I deprecate or retire an outdated Skill?* Milvus AI Quick Reference. <https://milvus.io/ai-quick-reference/how-do-i-deprecate-or-retire-an-outdated-skill>
- [77] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. 2026. Trace2Skill: Distill Trajectory-Local Lessons into Transferable Agent Skills. arXiv:2603.25158 [cs.AI] <https://arxiv.org/abs/2603.25158>
- [78] Zheng Nie, Ruolin Shen, Xinlei Yu, Bo Yin, Jiangning Zhang, and Xiaobin Hu. 2026. SkillGraph: Self-Evolving Multi-Agent Collaboration with Multimodal Graph Topology. arXiv:2604.17503 [cs.AI] <https://arxiv.org/abs/2604.17503>

- [79] Nous Research. 2026. Hermes Agent. <https://github.com/NousResearch/hermes-agent> Accessed: 2026-05-05.
- [80] OpenAI. 2025. Computer-Using Agent. <https://openai.com/index/computer-using-agent/>. Accessed: 2026-05-05.
- [81] OpenAI. 2026. Introducing Codex. <https://openai.com/index/introducing-codex/> Accessed: 2026-05-05.
- [82] OpenAI. 2026. Testing Agent Skills Systematically with Evals. <https://developers.openai.com/blog/eval-skills> Accessed: 2026-05-05.
- [83] OpenClaw. 2026. OpenClaw. <https://github.com/openclaw/openclaw> Accessed: 2026-05-05.
- [84] Pratik Panda. 2026. *Giving Your AI Agents Reliable Skills with the Agent Skills SDK*. Microsoft Developer Community Blog. <https://techcommunity.microsoft.com/blog/azuredevcommunityblog/giving-your-ai-agents-reliable-skills-with-the-agent-skills-sdk/4497074>
- [85] Xue Qin, Simin Luan, John See, Cong Yang, and Zhijun Li. 2026. Learning Without Losing Identity: Capability Evolution for Embodied Agents. arXiv:2604.07799 [cs.RO] <https://arxiv.org/abs/2604.07799>
- [86] Yubin Qu, Yi Liu, Tongcheng Geng, Gelei Deng, Yuekang Li, Leo Yu Zhang, Ying Zhang, and Lei Ma. 2026. Supply-Chain Poisoning Attacks Against LLM Coding Agent Skill Ecosystems. arXiv:2604.03081 [cs.CR] <https://arxiv.org/abs/2604.03081>
- [87] Christopher Rawles, Sarah Clinckemallie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. 2025. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. arXiv:2405.14573 [cs.AI] <https://arxiv.org/abs/2405.14573>
- [88] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. arXiv:2310.10501 [cs.CL] <https://arxiv.org/abs/2310.10501>
- [89] Samuel Rose. 2026. The Best AI Agent Skills Marketplaces in 2026 – Honest Comparison. Agensi Learn. <https://www.agensi.io/learn/best-ai-agent-skills-marketplaces-2026> Accessed: 2026-05-06.
- [90] Sampriti Saha and Pranav Hemanth. 2026. Skilldex: A Package Manager and Registry for Agent Skill Packages with Hierarchical Scope-Based Distribution. arXiv:2604.16911 [cs.AI] <https://arxiv.org/abs/2604.16911>
- [91] Gabriel Sarch, Lawrence Jang, Michael J. Tarr, William W. Cohen, Kenneth Marino, and Katerina Fragkiadaki. 2025. VLM Agents Generate Their Own Memories: Distilling Experience into Embodied Programs of Thought. arXiv:2406.14596 [cs.CV] <https://arxiv.org/abs/2406.14596>
- [92] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761 [cs.CL] <https://arxiv.org/abs/2302.04761>
- [93] David Schmotz, Sahar Abdelnabi, and Maksym Andriushchenko. 2025. Agent Skills Enable a New Class of Realistic and Trivially Simple Prompt Injections. arXiv:2510.26328 [cs.LG] <https://arxiv.org/abs/2510.26328>
- [94] David Schmotz, Luca Beurer-Kellner, Sahar Abdelnabi, and Maksym Andriushchenko. 2026. Skill-Inject: Measuring Agent Vulnerability to Skill File Attacks. arXiv:2602.20156 [cs.CR] <https://arxiv.org/abs/2602.20156>
- [95] Shuaik Shen, Wenduo Cheng, Mingqian Ma, Alistair Turcan, Martin Jinze Zhang, and Jian Ma. 2026. SKILLFOUNDRY: Building Self-Evolving Agent Skill Libraries from Heterogeneous Scientific Resources. arXiv:2604.03964 [cs.AI] <https://arxiv.org/abs/2604.03964>
- [96] Tianmin Shu, Caiming Xiong, and Richard Socher. 2017. Hierarchical and Interpretable Skill Acquisition in Multi-task Reinforcement Learning. arXiv:1712.07294 [cs.AI] <https://arxiv.org/abs/1712.07294>
- [97] Weijia Song, Jiashu Yue, and Zhe Pang. 2026. ABSTRAL: Automatic Design of Multi-Agent Systems Through Iterative Refinement and Topology Optimization. arXiv:2603.22791 [cs.AI] <https://arxiv.org/abs/2603.22791>
- [98] Weihang Su, Jianming Long, Qingyao Ai, Yichen Tang, Changyue Wang, Yiteng Tu, and Yiqun Liu. 2026. Skill Retrieval Augmentation for Agentic AI. arXiv:2604.24594 [cs.CL] <https://arxiv.org/abs/2604.24594>
- [99] Theodore R. Summers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. 2024. Cognitive Architectures for Language Agents. arXiv:2309.02427 [cs.AI] <https://arxiv.org/abs/2309.02427>
- [100] Yiqun Sun, Pengfei Wei, and Lawrence B. Hsieh. 2026. Don't Retrieve, Navigate: Distilling Enterprise Knowledge into Navigable Agent Skills for QA and RAG. arXiv:2604.14572 [cs.IR] <https://arxiv.org/abs/2604.14572>
- [101] Chenwei Tang, Lin Long, Xinyu Liu, Jingyu Xing, Zizhou Wang, Joey Tianyi Zhou, Jiawei Du, Liangli Zhen, and Jiancheng Lv. 2026. SAG-Agent: Enabling Long-Horizon Reasoning in Strategy Games via Dynamic Knowledge Graphs. arXiv:2510.15259 [cs.AI] <https://arxiv.org/abs/2510.15259>
- [102] Chen Tessler, Shahar Givony, Tom Zahavy, Daniel J. Mankowitz, and Shie Mannor. 2016. A Deep Hierarchical Approach to Lifelong Learning in Minecraft. arXiv:1604.07255 [cs.AI] <https://arxiv.org/abs/1604.07255>
- [103] Guiyao Tie, Jiawen Shi, Pan Zhou, and Lichao Sun. 2026. BadSkill: Backdoor Attacks on Agent Skills via Model-in-Skill Poisoning. arXiv:2604.09378 [cs.CR] <https://arxiv.org/abs/2604.09378>
- [104] Songjun Tu, Chengdong Xu, Qichao Zhang, Yaocheng Zhang, Xiangyuan Lan, Linjing Li, and Dongbin Zhao. 2026. Dynamic Dual-Granularity Skill Bank for Agentic RL. arXiv:2603.28716 [cs.AI] <https://arxiv.org/abs/2603.28716>

- [105] Georgios Tzifas and Hamidreza Kasaei. 2024. Lifelong Robot Library Learning: Bootstrapping Composable and Generalizable Skills for Embodied Control with Language Models. arXiv:2406.18746 [cs.RO] <https://arxiv.org/abs/2406.18746>
- [106] Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, and Shumin Deng. 2026. SkillX: Automatically Constructing Skill Knowledge Bases for Agents. arXiv:2604.04804 [cs.CL] <https://arxiv.org/abs/2604.04804>
- [107] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291 [cs.AI] <https://arxiv.org/abs/2305.16291>
- [108] Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, Xiaoxin Chen, and Honggang Qi. 2026. Skill-SD: Skill-Conditioned Self-Distillation for Multi-turn LLM Agents. arXiv:2604.10674 [cs.LG] <https://arxiv.org/abs/2604.10674>
- [109] Haoyu Wang, Zibo Xiao, Yedi Zhang, Christopher M. Poskitt, and Jun Sun. 2026. SafeClaw-R: Towards Safe and Secure Multi-Agent Personal Assistants. arXiv:2603.28807 [cs.CR] <https://arxiv.org/abs/2603.28807>
- [110] Jiayu Wang, Yifei Ming, Zixuan Ke, Shafiq Joty, Aws Albarghouthi, and Frederic Sala. 2026. SkillOrchestra: Learning to Route Agents via Skill Transfer. arXiv:2602.19672 [cs.AI] <https://arxiv.org/abs/2602.19672>
- [111] Jiong Xiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. 2026. Reinforcement Learning for Self-Improving Agent with Skill Library. arXiv:2512.17102 [cs.AI] <https://arxiv.org/abs/2512.17102>
- [112] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
- [113] Qianli Wang, Boyang Ma, Minghui Xu, and Yue Zhang. 2026. When Skills Lie: Hidden-Comment Injection in LLM Agents. arXiv:2602.10498 [cs.CR] <https://arxiv.org/abs/2602.10498>
- [114] Zimu Wang, Yuling Shi, Mengfan Li, Zijun Liu, Jie M. Zhang, Chengcheng Wan, and Xiaodong Gu. 2026. EffiSkill: Agent Skill Based Automated Code Efficiency Optimization. arXiv:2603.27850 [cs.SE] <https://arxiv.org/abs/2603.27850>
- [115] Zhaoyang Wang, Qianhui Wu, Xuchao Zhang, Chaoyun Zhang, Wenlin Yao, Fazle Elahi Faisal, Baolin Peng, Si Qin, Suman Nath, Qingwei Lin, Chetan Bansal, Dongmei Zhang, Saravan Rajmohan, Jianfeng Gao, and Huaxiu Yao. 2026. WebXSkill: Skill Learning for Autonomous Web Agents. arXiv:2604.13318 [cs.AI] <https://arxiv.org/abs/2604.13318>
- [116] Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. 2025. Inducing Programmatic Skills for Agentic Tasks. arXiv:2504.06821 [cs.CL] <https://arxiv.org/abs/2504.06821>
- [117] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. Agent Workflow Memory. arXiv:2409.07429 [cs.CL] <https://arxiv.org/abs/2409.07429>
- [118] Haolin Wu, Yuecheng Liu, Junyi Dong, Heng Zhang, Sitong Mao, Hesheng Wang, Weigang Wu, and Shunbo Zhou. 2025. ASCENT: Autonomous Skill Learning Toward Complex Embodied Tasks With Foundation Models. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*. 16752–16758. doi:10.1109/ICRA55743.2025.11127927
- [119] Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, and Botian Shi. 2025. EvolveR: Self-Evolving LLM Agents through an Experience-Driven Lifecycle. arXiv:2510.16079 [cs.CL] <https://arxiv.org/abs/2510.16079>
- [120] Xiyang Wu, Zongxia Li, Guangyao Shi, Alexander Duffy, Tyler Marques, Matthew Lyle Olson, Tianyi Zhou, and Dinesh Manocha. 2026. Co-Evolving LLM Decision and Skill Bank Agents for Long-Horizon Tasks. arXiv:2604.20987 [cs.AI] <https://arxiv.org/abs/2604.20987>
- [121] Yaxiong Wu and Yongyue Zhang. 2026. Agent Skills from the Perspective of Procedural Memory: A Survey. *TechRxiv* 2026, 0116 (2026). arXiv:<https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.176857932.25697838/v1> doi:10.36227/techrxiv.176857932.25697838/v1
- [122] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. 2024. OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. arXiv:2402.07456 [cs.AI] <https://arxiv.org/abs/2402.07456>
- [123] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. arXiv:2602.08234 [cs.LG] <https://arxiv.org/abs/2602.08234>
- [124] Tianle Xia, Lingxiang Hu, Yiding Sun, Ming Xu, Lan Xu, Siying Wang, Wei Xu, and Jie Jiang. 2026. GraSP: Graph-Structured Skill Compositions for LLM Agents. arXiv:2604.17870 [cs.CL] <https://arxiv.org/abs/2604.17870>
- [125] Wenjie Xiao, Xuehai Tang, Biyu Zhou, Songlin Hu, and Jizhong Han. 2026. RouteGuard: Internal-Signal Detection of Skill Poisoning in LLM Agents. arXiv:2604.22888 [cs.CR] <https://arxiv.org/abs/2604.22888>
- [126] Senwei Xie, Yuntian Zhang, Ruiping Wang, and Xilin Chen. 2026. Uni-Skill: Building Self-Evolving Skill Repository for Generalizable Robotic Manipulation. arXiv:2603.02623 [cs.RO] <https://arxiv.org/abs/2603.02623>

- [127] Yuquan Xie, Zaijing Li, Rui Shao, Gongwei Chen, Kaiwen Zhou, Yinchuan Li, Dongmei Jiang, and Liqiang Nie. 2025. Mirage-1: Augmenting and Updating GUI Agent with Hierarchical Multimodal Skills. arXiv:2506.10387 [cs.AI] <https://arxiv.org/abs/2506.10387>
- [128] Yiqing Xie, Emmy Liu, Gaokai Zhang, Nachiket Kotalwar, Shubham Gandhi, Sathwik Acharya, Xingyao Wang, Carolyn Rose, Graham Neubig, and Daniel Fried. 2026. Hybrid-Gym: Training Coding Agents to Generalize Across Tasks. arXiv:2602.16819 [cs.SE] <https://arxiv.org/abs/2602.16819>
- [129] Hanwen Xing, Haomin Zhuang, Xuandong Zhao, Yue Huang, Zhenheng Tang, and Xiangliang Zhang. 2026. Recipes for Agents: Understanding Skills and Their Open Questions. doi:10.13140/RG.2.2.11421.99045
- [130] Mengda Xu, Zhenjia Xu, Cheng Chi, Manuela Veloso, and Shuran Song. 2023. XSkill: Cross Embodiment Skill Discovery. arXiv:2307.09955 [cs.RO] <https://arxiv.org/abs/2307.09955>
- [131] Renjun Xu and Yang Yan. 2026. Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward. arXiv:2602.12430 [cs.MA] <https://arxiv.org/abs/2602.12430>
- [132] Yangjie Xu, Lujun Li, Lama Sleem, Niccolo Gentile, Yewei Song, Yiqun Wang, Siming Ji, Wenbo Wu, and Radu State. 2026. Agent Skill Framework: Perspectives on the Potential of Small Language Models in Industrial Environments. arXiv:2602.16653 [cs.AI] <https://arxiv.org/abs/2602.16653>
- [133] Zishan Xu, Yifu Guo, YUQUAN LU, Fengyu Yang, Zhiyuan Yao, Jiaye Lin, Ruyi Gong, and Lihua Cai. 2026. SkillEvo: An Experience Learning Framework with Reinforcement Learning for Skill Evolution. <https://openreview.net/forum?id=S1cIE9pe3k>
- [134] Yongjin Yang, Sinjae Kang, Juyong Lee, Dongjun Lee, Se-Young Yun, and Kimin Lee. 2025. Automated Skill Discovery for Language Agents through Exploration and Iterative Feedback. arXiv:2506.04287 [cs.AI] <https://arxiv.org/abs/2506.04287>
- [135] Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. 2026. AutoSkill: Experience-Driven Lifelong Learning via Skill Self-Evolution. arXiv:2603.01145 [cs.AI] <https://arxiv.org/abs/2603.01145>
- [136] Zonghan Yang, Shengjie Wang, Kelin Fu, Wenyang He, Weimin Xiong, Yibo Liu, Yibo Miao, Bofei Gao, Yejie Wang, Yingwei Ma, Yanhao Li, Yue Liu, Zhenxing Hu, Kaitai Zhang, Shuyi Wang, Huarong Chen, Flood Sung, Yang Liu, Yang Gao, Zhilin Yang, and Tianyu Liu. 2025. Kimi-Dev: Agentless Training as Skill Prior for SWE-Agents. arXiv:2509.23045 [cs.AI] <https://arxiv.org/abs/2509.23045>
- [137] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 [cs.CL] <https://arxiv.org/abs/2210.03629>
- [138] Anbang Ye, Qianran Ma, Jia Chen, Muqi Li, Tong Li, Fujiao Liu, Siqi Mai, Meichen Lu, Haitao Bao, and Yang You. 2025. SOP-Agent: Empower General Purpose AI Agent with Domain-Specific SOPs. arXiv:2501.09316 [cs.AI] <https://arxiv.org/abs/2501.09316>
- [139] Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. 2026. Meta Context Engineering via Agentic Skill Evolution. arXiv:2601.21557 [cs.AI] <https://arxiv.org/abs/2601.21557>
- [140] Qing Ye and Jing Tan. 2026. Agent Contracts: A Formal Framework for Resource-Bounded Autonomous AI Systems. arXiv:2601.08815 [cs.MA] <https://arxiv.org/abs/2601.08815>
- [141] Simon Yu, Gang Li, Weiyan Shi, and Peng Qi. 2026. PolySkill: Learning Generalizable Skills Through Polymorphic Abstraction. arXiv:2510.15863 [cs.CL] <https://arxiv.org/abs/2510.15863>
- [142] Haoqi Yuan, Chi Zhang, Hongcheng Wang, Feiyang Xie, Penglin Cai, Hao Dong, and Zongqing Lu. 2023. Skill Reinforcement Learning and Planning for Open-World Long-Horizon Tasks. arXiv:2303.16563 [cs.LG] <https://arxiv.org/abs/2303.16563>
- [143] Bo Yue, Shuqi Guo, Kaiyu Hu, Chujiao Wang, Benyou Wang, Kui Jia, and Guiliang Liu. 2025. Real-Time Verification of Embodied Reasoning for Generative Skill Acquisition. arXiv:2505.11175 [cs.RO] <https://arxiv.org/abs/2505.11175>
- [144] Renos Zabounidis, Yue Wu, Simon Stepputtis, Woojun Kim, Yuanzhi Li, Tom Mitchell, and Katia Sycara. 2026. SCALAR: Learning and Composing Skills through LLM Guided Symbolic Planning and Deep RL Grounding. arXiv:2603.09036 [cs.LG] <https://arxiv.org/abs/2603.09036>
- [145] Yunpeng Zhai, Shuchang Tao, Cheng Chen, Anni Zou, Ziqian Chen, Qingxu Fu, Shinji Mai, Li Yu, Jiaji Deng, Zouying Cao, Zhaoyang Liu, Bolin Ding, and Jingren Zhou. 2025. AgentEvolver: Towards Efficient Self-Evolving Agent System. arXiv:2511.10395 [cs.LG] <https://arxiv.org/abs/2511.10395>
- [146] Guijia Zhang, Shu Yang, Xilin Gong, and Di Wang. 2026. STARS: Skill-Triggered Audit for Request-Conditioned Invocation Safety in Agent Systems. arXiv:2604.10286 [cs.AI] <https://arxiv.org/abs/2604.10286>
- [147] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu. 2026. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. arXiv:2604.01687 [cs.AI] <https://arxiv.org/abs/2604.01687>
- [148] Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026. MemSkill: Learning and Evolving Memory Skills for Self-Evolving Agents. arXiv:2602.02474 [cs.CL] <https://arxiv.org/abs/2602.02474>

02474

- [149] Kaiyi Zhang, Ang Lv, Jinpeng Li, Yongbo Wang, Feng Wang, Haoyuan Hu, and Rui Yan. 2025. StepHint: Multi-level Stepwise Hints Enhance Reinforcement Learning to Reason. arXiv:2507.02841 [cs.AI] <https://arxiv.org/abs/2507.02841>
- [150] Xing Zhang, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He. 2026. Experience Compression Spectrum: Unifying Memory, Skills, and Rules in LLM Agents. arXiv:2604.15877 [cs.AI] <https://arxiv.org/abs/2604.15877>
- [151] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2024. A Survey on the Memory Mechanism of Large Language Model based Agents. arXiv:2404.13501 [cs.AI] <https://arxiv.org/abs/2404.13501>
- [152] Zhang Zhang, Shuqi Lu, Hongjin Qian, Di He, and Zheng Liu. 2026. AgentFactory: A Self-Evolving Framework Through Executable Subagent Accumulation and Reuse. arXiv:2603.18000 [cs.AI] <https://arxiv.org/abs/2603.18000>
- [153] Ziao Zhang, Kou Shi, Shiting Huang, Avery Nie, Yu Zeng, Yiming Zhao, Zhen Fang, Qishen Su, Haibo Qiu, Wei Yang, Qingnan Ren, Shun Zou, Wenxuan Huang, Lin Chen, Zehui Chen, and Feng Zhao. 2026. SkillFlow: Benchmarking Lifelong Skill Discovery and Evolution for Autonomous Agents. arXiv:2604.17308 [cs.AI] <https://arxiv.org/abs/2604.17308>
- [154] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. arXiv:2308.10144 [cs.LG] <https://arxiv.org/abs/2308.10144>
- [155] Xufeng Zhao, Cornelius Weber, and Stefan Wermter. 2024. Agentic Skill Discovery. arXiv:2405.15019 [cs.RO] <https://arxiv.org/abs/2405.15019>
- [156] Zhonghan Zhao, Wenhao Chai, Xuan Wang, Li Boyi, Shengyu Hao, Shidong Cao, Tian Ye, and Gaoang Wang. 2024. See and Think: Embodied Agent in Virtual Environment. arXiv:2311.15209 [cs.AI] <https://arxiv.org/abs/2311.15209>
- [157] Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivas, Gaowen Liu, Graham Neubig, and Yu Su. 2025. SkillWeaver: Web Agents can Self-Improve by Discovering and Honing Skills. arXiv:2504.07079 [cs.AI] <https://arxiv.org/abs/2504.07079>
- [158] Xinyue Zheng, Haowei Lin, Kaichen He, Zihao Wang, Zilong Zheng, and Yitao Liang. 2025. MCU: An Evaluation Framework for Open-Ended Game Agents. arXiv:2310.08367 [cs.AI] <https://arxiv.org/abs/2310.08367>
- [159] YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. 2026. SkillRouter: Skill Routing for LLM Agents at Scale. arXiv:2603.22455 [cs.LG] <https://arxiv.org/abs/2603.22455>
- [160] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. 2026. SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks. arXiv:2604.20087 [cs.CL] <https://arxiv.org/abs/2604.20087>
- [161] Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. 2026. Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering. arXiv:2604.08224 [cs.SE] <https://arxiv.org/abs/2604.08224>
- [162] Huichi Zhou, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, Menglong Zhang, Yihang Chen, Jinsong Li, Runyu Yang, Qiangbin Liu, Xinlei Yu, Jianmin Zhou, Na Wang, Chunyang Sun, and Jun Wang. 2026. Memento-Skills: Let Agents Design Agents. arXiv:2603.18743 [cs.AI] <https://arxiv.org/abs/2603.18743>
- [163] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854 [cs.AI] <https://arxiv.org/abs/2307.13854>
- [164] Jiaying Zhu, Lyuye Zhang, Wenbo Guo, and Yang Liu. 2026. SkillClone: Multi-Modal Clone Detection and Clone Propagation Analysis in the Agent Skill Ecosystem. arXiv:2603.22447 [cs.SE] <https://arxiv.org/abs/2603.22447>
- [165] Haomin Zhuang, Hanwen Xing, and Xiangliang Zhang. 2026. AgentClick: A Skill-Based Human-in-the-Loop Review Layer for Terminal AI Agents. arXiv:2604.16520 [cs.HC] <https://arxiv.org/abs/2604.16520>